

# Master Thesis

## Comparison of the Different Solutions of a Mobile Robot Outdoor Localization using the ROS 2 Components

to attain the academic degree  
Master of Science in Mechatronics

submitted by:

**Gaurav Kapoor**  
<gk-192139@rwu.de>

December 27, 2021

First Reviewer: Prof. Dr. rer. nat. Stefan Elser <stefan.elser@rwu.de>  
Second Reviewer: Prof. Dr. rer. nat. Wolfgang Ertel <ertel@rwu.de>



# Contents

|          |                                                                                                                            |           |
|----------|----------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Declaration of Authorship</b>                                                                                           | <b>8</b>  |
| <b>2</b> | <b>Abstract</b>                                                                                                            | <b>9</b>  |
| <b>3</b> | <b>Acknowledgements</b>                                                                                                    | <b>10</b> |
| <b>4</b> | <b>Introduction</b>                                                                                                        | <b>11</b> |
| <b>5</b> | <b>Related Work</b>                                                                                                        | <b>12</b> |
| <b>6</b> | <b>Method</b>                                                                                                              | <b>13</b> |
| 6.1      | Analysis of the Requirements . . . . .                                                                                     | 13        |
| 6.1.1    | Robot and the Sensor Setup . . . . .                                                                                       | 14        |
| 6.1.2    | Sensor Drivers . . . . .                                                                                                   | 20        |
| 6.2      | Potential Approaches and Problems . . . . .                                                                                | 26        |
| 6.2.1    | Point Cloud Map Making and the 3D Localization Challenges Setup . . . . .                                                  | 26        |
| 6.2.2    | rosvbag2 Dropping Messages . . . . .                                                                                       | 30        |
| 6.2.3    | Frame Transformations in ROS . . . . .                                                                                     | 30        |
| 6.2.4    | Point Cloud Map Making . . . . .                                                                                           | 32        |
| 6.2.5    | 2D Occupancy Grid Map . . . . .                                                                                            | 36        |
| 6.2.6    | Point Cloud Map PCD File Output from the Google Cartographer 3D SLAM<br>Algorithm . . . . .                                | 36        |
| 6.2.7    | PCD File . . . . .                                                                                                         | 37        |
| 6.2.8    | Comparison of the Quality of the 3D Localization Algorithms with the Ground<br>Truth RTK GNSS Receiver Positions . . . . . | 38        |
| 6.2.9    | GNSS Data Coordinate Frames . . . . .                                                                                      | 38        |
| 6.2.10   | Visualization of the Robot Model in RViz . . . . .                                                                         | 40        |
| 6.2.11   | 3D Localization Algorithms Discussed, Evaluated and Compared . . . . .                                                     | 41        |
| <b>7</b> | <b>Results</b>                                                                                                             | <b>46</b> |
| 7.1      | Google Cartographer 3D SLAM Quality . . . . .                                                                              | 46        |
| 7.1.1    | Trajectories Comparisons . . . . .                                                                                         | 46        |
| 7.1.2    | Absolute Pose Errors . . . . .                                                                                             | 48        |
| 7.2      | 3D Localization Challenges . . . . .                                                                                       | 51        |
| 7.2.1    | No Obstacles . . . . .                                                                                                     | 51        |
| 7.2.2    | High Velocity . . . . .                                                                                                    | 58        |

|       |                          |    |
|-------|--------------------------|----|
| 7.2.3 | Fog . . . . .            | 65 |
| 7.2.4 | Pitch . . . . .          | 71 |
| 7.2.5 | Roll . . . . .           | 76 |
| 7.2.6 | Overall Winner . . . . . | 82 |

|          |                                    |           |
|----------|------------------------------------|-----------|
| <b>8</b> | <b>Conclusions and Future Work</b> | <b>83</b> |
|----------|------------------------------------|-----------|



# List of Figures

|      |                                                                                                                                                                                  |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 6.1  | MATTRO ROVO 2 - Performance . . . . .                                                                                                                                            | 14 |
| 6.2  | MATTRO ROVO 2 Radio Remote Control . . . . .                                                                                                                                     | 15 |
| 6.3  | PEAK-System PCAN-USB Device [1] . . . . .                                                                                                                                        | 15 |
| 6.4  | Logitech F710 Wireless Gamepad . . . . .                                                                                                                                         | 16 |
| 6.5  | Lenovo ThinkPad W541 with the Docking Station . . . . .                                                                                                                          | 17 |
| 6.6  | Ouster OS1-128 3D Lidar Sensor . . . . .                                                                                                                                         | 17 |
| 6.7  | Emlid Reach M+ RTK GNSS Receiver . . . . .                                                                                                                                       | 18 |
| 6.8  | Swift Navigation Mini Survey Antenna GPS500 . . . . .                                                                                                                            | 19 |
| 6.9  | rqt_graph of the Dataset Recording . . . . .                                                                                                                                     | 27 |
| 6.10 | Frequencies of the Sensor Data as Recorded in a Bag at an Instant . . . . .                                                                                                      | 28 |
| 6.11 | Slope Used for the Pitch and the Roll 3D Localization Challenge . . . . .                                                                                                        | 30 |
| 6.12 | Frame Transformations Tree . . . . .                                                                                                                                             | 32 |
| 6.13 | Raw Input Point Cloud . . . . .                                                                                                                                                  | 34 |
| 6.14 | Voxelized Input Point Cloud . . . . .                                                                                                                                            | 34 |
| 6.15 | rqt_graph of the Google Cartographer 3D SLAM Algorithm . . . . .                                                                                                                 | 34 |
| 6.16 | Point Cloud Map Made by the Google Cartographer 3D SLAM Algorithm . . . . .                                                                                                      | 36 |
| 6.17 | Real Map Image from Google Maps . . . . .                                                                                                                                        | 40 |
| 6.18 | 2D Occupancy Grid Map Output from the Google Cartographer pbstream_to_ros_map Component . . . . .                                                                                | 40 |
| 6.19 | Visualization of the Robot Model along with its Frames in RViz . . . . .                                                                                                         | 41 |
| 6.20 | 3D NDT Scan-Matching in the Autoware.Auto 3D NDT Localization Algorithm . . . . .                                                                                                | 42 |
| 6.21 | rqt_graph of the Autoware.Auto 3D NDT Localization Algorithm . . . . .                                                                                                           | 42 |
| 6.22 | pcl_localization_ros2 3D Localization Algorithm Node Showing its Published and Subscribed Topics . . . . .                                                                       | 44 |
| 7.1  | Map Comparison of the Cartographer 3D SLAM and the Ground Truth Trajectory . . . . .                                                                                             | 47 |
| 7.2  | Axis-Wise Comparison of the Cartographer 3D SLAM and the Ground Truth Trajectory . . . . .                                                                                       | 47 |
| 7.3  | Axis-Wise Comparison of the Cartographer 3D SLAM with the Ground Truth Trajectory after their Rotation by Thirty Degrees about the Z-Axis Anti-Clockwise . . . . .               | 48 |
| 7.4  | Raw Distribution of the 3D APEs in the Cartographer 3D SLAM Trajectory . . . . .                                                                                                 | 48 |
| 7.5  | Raw Distribution of the 2D APEs in the Cartographer 2D SLAM Trajectory . . . . .                                                                                                 | 49 |
| 7.6  | Map Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the No Obstacles 3D Localization Challenge . . . . . | 51 |

|      |                                                                                                                                                                                                                                                                |    |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 7.7  | Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the No Obstacles 3D Localization Challenge . . . . .                                                                         | 52 |
| 7.8  | Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories after their Rotation by Thirty Degrees about the Z-Axis Anti-Clockwise in the No Obstacles 3D Localization Challenge . . . . .  | 52 |
| 7.9  | Raw Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the No Obstacles 3D Localization Challenge . . . . .                                                                                                                   | 53 |
| 7.10 | Raw Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the No Obstacles 2D Localization Challenge . . . . .                                                                                                                   | 54 |
| 7.11 | Boxplot of the Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the No Obstacles 3D Localization Challenge . . . . .                                                                                                        | 55 |
| 7.12 | Boxplot of the Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the No Obstacles 2D Localization Challenge . . . . .                                                                                                        | 56 |
| 7.13 | Map Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the High Velocity 3D Localization Challenge . . . . .                                                                              | 58 |
| 7.14 | Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the High Velocity 3D Localization Challenge . . . . .                                                                        | 59 |
| 7.15 | Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories after their Rotation by Thirty Degrees about the Z-Axis Anti-Clockwise in the High Velocity 3D Localization Challenge . . . . . | 60 |
| 7.16 | Raw Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the High Velocity 3D Localization Challenge . . . . .                                                                                                                  | 61 |
| 7.17 | Raw Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the High Velocity 2D Localization Challenge . . . . .                                                                                                                  | 62 |
| 7.18 | Boxplot of the Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the High Velocity 3D Localization Challenge . . . . .                                                                                                       | 63 |
| 7.19 | Boxplot of the Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the High Velocity 2D Localization Challenge . . . . .                                                                                                       | 63 |
| 7.20 | Map Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Fog 3D Localization Challenge . . . . .                                                                                        | 66 |
| 7.21 | Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Fog 3D Localization Challenge . . . . .                                                                                  | 66 |
| 7.22 | Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories after their Rotation by Thirty Degrees about the Z-Axis Anti-Clockwise in the Fog 3D Localization Challenge . . . . .           | 67 |
| 7.23 | Raw Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Fog 3D Localization Challenge . . . . .                                                                                                                            | 67 |
| 7.24 | Raw Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the Fog 2D Localization Challenge . . . . .                                                                                                                            | 68 |

|      |                                                                                                                                                                                 |    |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 7.25 | Boxplot of the Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Fog 3D Localization Challenge . . . . .                                  | 68 |
| 7.26 | Boxplot of the Distributions of the 2D APEs for All the Three Tested 3D Localization Algorithms in the Fog 2D Localization Challenge . . . . .                                  | 69 |
| 7.27 | Map Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Pitch 3D Localization Challenge                 | 71 |
| 7.28 | Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Pitch 3D Localization Challenge . . . . . | 72 |
| 7.29 | Raw Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Pitch 3D Localization Challenge . . . . .                                           | 72 |
| 7.30 | Raw Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the Pitch 2D Localization Challenge . . . . .                                           | 73 |
| 7.31 | Boxplot of the Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Pitch 3D Localization Challenge . . . . .                                | 74 |
| 7.32 | Boxplot of the Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the Pitch 2D Localization Challenge . . . . .                                | 74 |
| 7.33 | Map Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Roll 3D Localization Challenge .                | 77 |
| 7.34 | Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Roll 3D Localization Challenge            | 77 |
| 7.35 | Raw Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Roll 3D Localization Challenge . . . . .                                            | 78 |
| 7.36 | Raw Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the Roll 2D Localization Challenge . . . . .                                            | 78 |
| 7.37 | Boxplot of the Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Roll 3D Localization Challenge . . . . .                                 | 79 |
| 7.38 | Boxplot of the Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the Roll 2D Localization Challenge . . . . .                                 | 80 |

# 1 Declaration of Authorship

I hereby declare that this master thesis is entirely the result of my own work except where otherwise indicated. I have only used the resources given in the list of references.

Gauran

88214 Ravensburg, December 27, 2021

## 2 Abstract

The aim of this master thesis was to find and evaluate the already available state-of-the-art 3D lidar-based localization algorithms in ROS 2. Localization is carried out to find the pose, i.e., the position and the orientation of a mobile robot with respect to the earth or a local map. This pose is used in robot navigation applications like agricultural and service robotics. ROS 2 [2] is the successor of ROS 1 [3], where ROS is the abbreviation of the Robot Operating System.

Localization is of two types - absolute localization and relative localization. In absolute localization, the pose of a robot is determined with respect to the earth coordinate frame in the form of LLA (latitude, longitude and altitude) or ECEF (earth-centered, earth-fixed) coordinates. This is performed by using GNSS (global navigation satellite system) receivers. In relative localization, the pose of a robot with respect to a known map is determined. In this master thesis, relative localization is performed where the pose of a robot with respect to a map is determined. This is performed by sensors like lidar, IMU (inertial measurement unit) and the wheel encoders. The environment in which the point cloud map is made and all the 3D localization challenges are tested is the InMach's [4] parking lot. For comparison of the 3D localization algorithms with the ground truth, the RTK (real-time kinematic) GNSS positions are used.

It is also shown in 7, that the GNSS receiver outputs very noisy data when the robot is driven near to the buildings. Therefore in places where the GNSS positions are not accurate, or for places deprived of GNSS signals like a tunnel or indoor environments, relative localization is required. The sensors used for relative localization like lidar, IMU and the wheel encoders do not require a globally referenced position and therefore not do not face the problems which are faced by the GNSS receivers as described in 6.2.1. Also, a lidar sensor is able to perceive the environment and measures the distances very accurately, as a laser travels at the speed of light.

### **3 Acknowledgements**

I would like to thank Dr. Boris Kluge and Dr. Siegfried Hochdorfer from InMach Intelligente Maschinen GmbH for providing me the opportunity to write my master thesis in their company. I would also like to thank them for providing me continuous help and support with my questions and doubts. I would like to thank my university professors, Dr. rer. nat. Stefan Elser and Dr. rer. nat. Wolfgang Ertel for supervising my master thesis and for their help and support.

## 4 Introduction

There are already existing 2D lidar-based localization algorithms in ROS 2, like nav2\_amcl [5] and slam\_toolbox [6] [7]. They are used to determine the 2D pose of a robot, i.e., x, y and yaw with respect to a map. But nowadays the robots are not only used indoors, but outdoors also in places for example, agricultural farms where there is an uneven ground for which a 2D localization is not enough. Here the person needs to know the robot's 6D pose consisting of x, y, z, roll, pitch and yaw with respect to a map, as now the robot has six degrees of freedom. Therefore in this master thesis, only the 3D localization algorithms are discussed, evaluated and compared.

For relative localization, a map is required on which a robot will localize itself relative to that map. A map can be created using a mapping or a SLAM (simultaneous localization and mapping) algorithm as described in 6.2.4. In this master thesis, the 3D localization algorithms based on the process of scan-matching [8] are discussed, evaluated and compared. Scan-matching in a localization algorithm is the process of aligning the current lidar scan with the nearest map scan (reference scan) based on the current pose predicted by its motion model [9] with respect to its previous pose. Then the coordinate frame transformation between the previous pose of the robot and the current pose of the robot is computed. The motion model is discussed in 6.1. In this master thesis the word frame will be used synonymously for a coordinate frame.

In ROS, the map used for 2D localization is usually in the form of a PGM (portable gray map) image file [10] where its meta-data [11] is written in a YAML file [12]. More information about this map is mentioned in 6.2.5. But in the case of 3D localization, a map is in the form of a point cloud which is usually a PCD (point cloud data) [13] file. More information about a PCD file is discussed in 6.2.6.

In ROS, there are nodes which are an executable to publish or subscribe messages to or from a topic respectively. The ROS version used in this application is the ROS 2 Foxy Fitzroy [14]. After discussing the method used for making the point cloud map and the 3D localization challenges in 6, the winner of all the 3D localization challenges is decided in 7. Also in 7, an overall winner of all the 3D localization challenges is decided. The overall winner of all the 3D localization challenges is possibly the best 3D localization algorithm for the used sensor setup. The word possibly is used as with more parameter tuning of a 3D localization algorithm or with additional sensor data input if it can be used by a 3D localization algorithm, the results could change.

## 5 Related Work

For the relative localization, the 3D localization algorithms used in this master thesis are based on the following approaches:

**ICP (Iterative Closest Point)** In this approach, the input lidar scans are aligned with its nearest lidar scans in the map by considering the scans as points. So for determining the pose of a robot the frame transformation is found out which minimizes the least squared error between the input scan and its nearest map scan. This frame transformation in fact is the pose of the robot with respect to that scan in the map. This process is repeated until the robot is driven in the map and the frame transformation matrices are pre-multiplied to determine the pose of the robot with respect to the origin of the map. Although the Google Cartographer SLAM algorithm [15] used in this master thesis does not inform on which algorithm its localization mode is based on, the author of this master thesis strongly believes that it is based on an approach similar to the ICP algorithm. This is because it also aligns the input scans and its nearest map scans by the least-squares method. More about the working of this SLAM algorithm is given in 6.2.4.

**NDT (Normal Distributions Transform)** In this approach the map is divided into grids. Each cell in the grid, which is a square (or a cube, for 3D) containing at least three points is modeled as a normal distribution as mentioned in [16] and [17]. For each cell (normal distribution) the mean of its probability density function is the mean of the points in that cell and its covariance is the covariance of the points in that cell. Then the points on the input lidar scan are aligned with these grid cells (nearest map scan) by maximizing the sum of all the probability density functions. The parameter which maximizes the sum of the probability density functions is the frame transformation that aligns the input scans with the map scans. The initialization estimate of this parameter is provided by the odometry data. The 3D localization algorithms based on the NDT approach used in this master thesis are the Autoware.Auto 3D NDT localization algorithm [18] [19] and the pcl\_localization\_ros2 3D PCL NDT localization algorithm [20]. More information about these algorithms is given in 6.2.11. The odometry data is discussed in 6.1.



# 6 Method

## 6.1 Analysis of the Requirements

The minimum sensor setup required for a lidar-based scan-matching localization algorithm to perform the scan-matching is a lidar sensor. Further sensor data inputs like the odometry, IMU and the GNSS data can be used to improve the quality of the localization provided by the localization algorithms. Odometry is the estimate of the 2D pose of a robot with respect to an initial pose which is computed by integrating the velocity increments received from the wheel encoders of the robot. But it accumulates errors over the time due to the reasons mentioned in 6.2.11. The odometry data is used to guess the initial pose of the robot. This is because it is hard to guess the initial pose of the robot with just a lidar sensor. The IMU data is used by the localization algorithms in the following ways:

1. The IMU data is used to estimate the orientation of the robot from its three-axis accelerometer, three-axis magnetometer and three-axis gyroscope. The roll, pitch and yaw angles can be estimated by integrating the angular velocities from the three-axis gyroscope in the IMU. But the measured angles accumulate drift over time as the angular velocities are discretely integrated at the frequency of the gyroscope data. For example in this application the frequency of the gyroscope data was 100 hertz. Also if the gyroscope data has bias and is noisy it accumulates errors due to its integration over the time also called angular random walk. The bias in a gyroscope is defined as a constant offset from the non-zero angular velocity output even when it is at rest. But the drift in the roll and the pitch angles can be corrected by estimating them from the accelerations provided by the three-axis accelerometer. This is because the roll and the pitch angles are referenced by the gravity vector pointing downwards and therefore do not drift. The drift in the yaw angles can be corrected by using the three-axis magnetometer providing the magnetic field values, as it provides the yaw angles referenced with the earth's magnetic north pole.
2. The odometry of the robot obtained by integrating the velocity increments received from the wheel encoders of the robot, is only 2D consisting of x, y and yaw of the robot. But the outdoor environments are 3D, and therefore require the 6D odometry containing the x, y, z, roll, pitch and yaw of the robot. Therefore the 2D odometry is fused with the IMU data to get the 3D odometry. This fusion is performed by the motion model component of the localization algorithms to estimate the pose of the robot with respect to its previous pose. In the Google Cartographer SLAM algorithm, the motion model is modeled in its pose

extrapolator component [21].

3. The IMU data is used to initially guess the orientation of the lidar scans for the scan-matching. This also reduces the complexity in the scan-matching as mentioned in [22].
4. The lidar sensor which is mounted on the robot also moves and rotates during each full revolution lidar scan. This is due to its rotating mirror which reflects the laser beam of light from a laser-range finder to calculate the distance between the lidar sensor and the objects. This causes the linear and the angular distortion in the lidar scans which is called as the lidar distortion. The accelerations from its three-axis accelerometer are double integrated to estimate the change in position of the robot to correct for the linear distortion in the lidar scans. The angular velocities from its three-axis gyroscope are integrated to estimate the change in rotation of the robot to correct the angular distortion in the lidar scans. Its implementation can be seen in [23].
5. Due the presence of a three-axis accelerometer inside an IMU, it is also used for the alignment of the -z-axis of the map with the gravity vector.

The GNSS data is used as an input in the localization algorithms to correct the relative localization through a globally referenced frame.

## 6.1.1 Robot and the Sensor Setup

### Robot



Figure 6.1: MATTRO ROVO 2 - Performance



Figure 6.2: MATTRO ROVO 2 Radio Remote Control

The robot used was MATTRO ROVO 2 - Performance [24]. Its picture is shown in 6.1. It is manufactured by MATTRO GmbH which is based in Austria. It is a tracked wheel robot weighing 290 kg with a top speed of 30 km/h. The robot manufacturer, i.e., Mattro has provided a radio remote control to drive the robot which also has an emergency stop button. Its picture is shown in 6.2. It also has a CAN (controller area network) bus through which velocity commands can be sent and received. The velocity commands were sent through the CAN bus, because the CAN protocol cannot be activated when the robot is driven by the remote control provided by Mattro.



Figure 6.3: PEAK-System PCAN-USB Device [1]

The bit-rate of the CAN communication is fixed at 500 kBit/s. The CAN bus is connected to the PC through the USB port using the PEAK-System PCAN-USB device and a 120 Ohm terminating resistor. The PEAK-System PCAN-USB device, the 120 Ohm terminating resistor and the MATTRO CAN bus cable used is shown in figure 6.3. An aluminum frame was constructed on the robot for safely mounting the sensors, the notebook PC and the inverter which was used for charging the notebook PC and giving power to the lidar sensor. The inverter took power from the robot's battery. The robot's battery gives power to its left and right electric motors and its electrical components including the CAN bus.

## Joystick



Figure 6.4: Logitech F710 Wireless Gamepad

For sending the velocity commands to the robot, the Logitech F710 wireless gamepad [25] was used. Its picture is shown in 6.4. The vertical axis of the gamepad's left analog stick was used to send the translational velocity commands to the robot and its right analog stick's horizontal axis was used for sending it the angular velocity commands. This was because of the reason that if a person wants to drive the robot in a straight line, they can use just the gamepad's left analog stick and if they want to rotate the robot at a place they can use just its right analog stick. If only one gamepad's analog stick was used to send the translational and rotational velocity commands to the robot then the following problems would arise:

1. There would be mostly be a rotational velocity command when a person wants to send only a translational velocity command, and a translational velocity command when a person wants to send only a rotational velocity command to the robot. This is because the gamepad's analog sticks can rarely be rotated purely vertically or horizontally due to human errors, like a person does not look at the gamepad when driving the robot, but only looks at the robot and its environments. Even if a person looks at the gamepad while driving the robot, they would never be sure that they are rotating the joystick perfectly in a straight line horizontally or vertically.
2. The unnecessary angular movement of the robot when a person wants the robot to drive it in a straight line, causes the robot to skid as it is a tracked wheel robot. This skidding causes tangential acceleration in the IMU depending on its distance from the instantaneous center of rotation of the robot, corrupting its linear acceleration measurements.

## Notebook PC

The notebook PC used was a Lenovo ThinkPad W541 [26] with a docking station. Its picture is shown in 6.5. This PC's specifications are:



Figure 6.5: Lenovo ThinkPad W541 with the Docking Station

**Memory** 16GB RAM

**Processor** Intel® Core™ i7-4810MQ CPU @ 2.80GHz × 8 [27]

**Storage** 500GB SATA SSD

**Graphics** NVIDIA® Quadro® K2100M 2GB memory [28]

**Operating System** Ubuntu 20.04.3 LTS [29]

### **Lidar Sensor and the IMU**

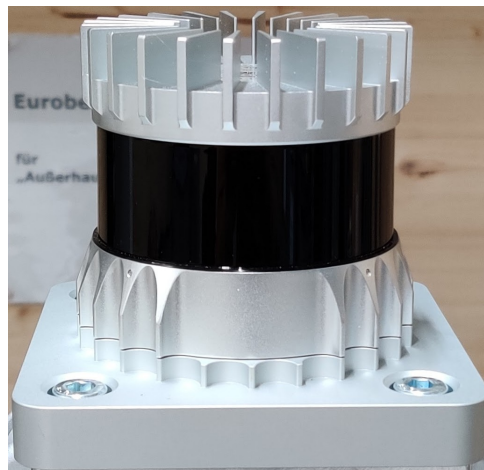


Figure 6.6: Ouster OS1-128 3D Lidar Sensor

The lidar sensor used was the Ouster OS1-128 3D lidar sensor [30]. Its picture is shown in 6.6. It has 360 degrees horizontal and 45 degrees vertical (-22.5 degrees to +22.5 degrees) field of view. It was mounted on the aluminum frame and interfaces to the PC through a gigabit Ethernet interface via a standard RJ45 connector. It was mounted at a sufficiently high height so that its horizontal and vertical field of view are not blocked by the robot's body and the aluminum frame. It has a maximum range of 120 m and has 128 vertical layers. It also has an IMU inside it which includes a three-axis gyroscope and a three-axis accelerometer. A three-axis gyroscope provides an angular velocity about its x, y and z axis and a three-axis accelerometer provides linear acceleration in its x, y and z axis, both with respect to an inertial frame of reference expressed in the IMU sensor frame [31].

## GNSS Receiver



Figure 6.7: Emlid Reach M+ RTK GNSS Receiver

The GNSS receiver used was Emlid Reach M+ [32] RTK GNSS receiver. Its picture is shown in 6.7. It is a single-band GNSS receiver which receives satellite signals on one frequency band. The GNSS receiver receives GNSS signals through a GNSS antenna. For a GNSS antenna to receive RTK signals, it should have a clear sky view thirty degrees above the horizon with no obstacles like trees, buildings, etc., blocking its view. In RTK GNSS receivers there is a base station and a rover which receive GNSS signals. The base station is static and sends correction signals to the rover. The rover is basically the GNSS receiver which is mobile and in this application is placed on the robot.

Swift Navigation Mini Survey Antenna GPS500 [33] was used as the GNSS antenna which was mounted on the aluminum frame. Its picture is shown in 6.8. This GNSS antenna receives GPS (Global Positioning System), GLONASS (Global Navigation Satellite System), BeiDou, Galileo, SBAS (Satellite-based augmentation systems) and QZSS (Quasi-Zenith Satellite System) signals. The GNSS receiver interfaces to the PC through Ethernet over USB interface via a USB 3.0 port. It also gets power from the same USB 3.0 port through which it interfaces to the PC. For configuring, logging and visualizing the GNSS data there is the Reach Panel, which is a web-based app provided by the GNSS receiver manufacturer, i.e., Emlid. There are three types of solution status in the Reach Panel, which are single, float and fix. In the single solution status, the GNSS receiver provides the position which it receives from the satellites with a precision in meters (two meters





Figure 6.8: Swift Navigation Mini Survey Antenna GPS500

to four meters). In the float solution status, the corrections from the base station are applied and the precision is less than a meter. After some time when the position becomes precise to centimeter-level, the solution status changes to fix.

In this application, the RTK base corrections were provided from Satellitenpositionierungsdienst (SAPOS®) Baden-Württemberg [34]. It took about 15 minutes to get the RTK fix signal since the time the GNSS receiver was powered on, on a clear sky in the evening at about 16:00 on 25<sup>th</sup> September 2021. For receiving the GNSS receiver positions to the PC through the USB, the serial communication interface is selected in the position output tab of the Reach Panel. Then in the device options the mode is set to USB-to-PC, baud rate to 9600 and data output format to NMEA.

The basics of the how the position of a GNSS receiver is computed using the satellites is as follows:

1. The position of the satellites is already known and their distance from a GNSS receiver is computed by subtracting the time difference of the signals transmitted and received and multiplying it with the speed of light.
2. The time difference of the arrival of the signals from a pair of satellites is constant, and therefore the locus of a GNSS receiver is a hyperboloid, with the two satellites at its foci.
3. Using a third satellite provides another time difference of arrival, and therefore another hyperboloid. The locus of the intersection of these two hyperboloids is curve.
4. Using a fourth satellite provides another time difference of arrival, and therefore another hyperboloid. The locus of the intersection of these three hyperboloids is one or two points, on one of which a GNSS receiver is located.
5. More than four satellites can be used to improve the accuracy in the positioning of a GNSS

receiver.

## 6.1.2 Sensor Drivers

The ROS 2 drivers of the sensors were used in this application. The following sensor drivers were used in this application:

### Joystick Driver

For the joystick, the ROS 2 joystick driver [35] was used. It publishes Joy messages [36] over the /joy topic. A Joy message contains the following information:

**Timestamp** It is the time at which a Joy message is published to a topic. It is the ROS time which is synchronized with the system time.

**Button values** These are binary values which denote one when a button is pressed on the joystick, and zero when not pressed.

**Analog stick axis values** The default values of this axis are from -1.0 to 1.0, which can be scaled depending upon the desired velocity of the robot. The default scaling was set to 1.0 for the maximum linear and angular velocity of the robot to be  $\pm 1.0$  meters per second and  $\pm 1.0$  radians per second respectively. The forward and the left movement of the analog stick is considered as a positive movement in the axis value and vice-versa.

The following parameters for this driver were changed from their default settings:

**deadzone** It is the minimum displacement of the analog stick needed from its center for the driver to consider a non zero value in the axis displacement. It was set to zero for the robot to receive a velocity command even with the slightest displacement of the analog stick.

**autorepeat\_rate** It is the rate at which the joystick publishes Joy messages when the state of the joystick is constant. It is by default twenty hertz. It was set to 100 hertz to match with the frequency of the velocity increments received from the wheel encoders of the robot.

**coalesce\_interval\_ms** It is the number of milliseconds for the driver to wait for publishing a Joy message after the analog stick of the joystick has been displaced. It is by default one millisecond but was set to zero milliseconds to instantly publish a Joy message after the analog stick is displaced.



## Odometry Driver

The ROS 2 C++ driver [37] was developed for sending and receiving the velocity commands through the CAN bus in the robot. The velocity increments received are used to compute the 2D odometry of the robot. This is because the velocity increments received from the wheel encoders of the robot are received after the robot executes the motion command (sent velocity increments) and therefore is accurate. But the velocity increments sent to the robot is just a wish that the robot receives the sent velocity command and therefore is not accurate. This odometry driver publishes Odometry messages [38] over the /odom topic. An Odometry message contains the following information:

**Timestamp** It is the time at which an Odometry message is published to a topic. It is the ROS time which is synchronized with the system time.

**Parent Frame** It is the parent frame relative to which the odometry of the robot is expressed in the base\_link frame. In this application it is called as the odom frame.

**Child Frame** It is the frame attached to the base of the robot and in this application is called as the base\_link frame. The odom and the base\_link frames are in accordance with [39].

**Pose with Covariance** It is in the form of a PoseWithCovariance message [40]. A PoseWithCovariance message contains the following information:

**Pose** It is the pose of the robot (child frame) in reference to the parent frame as a Pose message [41]. A Pose message contains the pose of a robot as an x, y, z position in meters as a Point message [42] and its orientation in the form of a quaternion as a Quaternion message [43]. In this application it was the 2D pose of the robot (base\_link frame) with respect to the odom frame.

**Pose Covariance** It is covariance of that pose as a 6x6 diagonal matrix in the row-major order in the x, y, z axis in meters squared and about the x, y, z axis in radians squared. Row-major means that the data is stored row-wise. In this application all the values in this 6x6 covariance matrix were zero which denotes that the covariance of the 2D odometry pose was unknown.

**Velocity with Covariance** It is in the form of a TwistWithCovariance message [44]. A TwistWithCovariance message contains the following information:

**Velocity** It is the velocity of the robot (child frame) with respect to an inertial frame of reference expressed in the child frame as a Twist message [45]. A Twist message contains the linear velocity in meters per second in the x, y and z-axis and the angular velocity in radians per second about the x, y, z axis both as a Vector3 message [46].

In this application the velocity of the robot was in the linear x-axis and about the angular z-axis only. This was because the robot used was a differential tracked-wheel drive robot which had non-holonomic constraints preventing it to move sideways.

**Velocity Covariance** It is the covariance of that velocity as a 6x6 diagonal matrix in the row-major order in the x, y, z axis in meters squared per seconds squared and about the x, y, z axis in radians squared per seconds squared. In this application all the values in this 6x6 covariance matrix were zero which denotes that the covariance of the odometry velocity was unknown.

The odometry is modeled considering the robot as a differential wheeled drive robot as it has two caterpillar tracks mimicking a robot's wheels which has a differential drive. The mathematical formulas used for computing the odometry from the velocity increments received from the wheel encoders in SI units are given in 1.

---

**Algorithm 1** Computation of the Odometry from the Received Velocity Increments

---

```

 $v_l, v_r, v, \omega, \Delta s, \Delta \theta, \Delta x, \Delta y, x$  and  $y \leftarrow 0$ 
 $W \leftarrow$  wheelbase of the robot
while velocity increments are received do
     $\Delta t \leftarrow$  time difference between current and the previously received velocity increments
     $v_l \leftarrow$  currently received left track velocity
     $v_r \leftarrow$  currently received right track velocity
    Linear velocity,  $v \leftarrow (v_l + v_r) \div 2$ 
    Angular velocity,  $\omega \leftarrow (v_r - v_l) \div W$ 
    Incremental linear displacement,  $\Delta s \leftarrow v \times \Delta t$ 
    Incremental angular displacement,  $\Delta \theta \leftarrow \omega \times \Delta t$ 
    Incremental linear displacement in the x-axis,  $\Delta x \leftarrow \Delta s \times \cos(\theta + \Delta \theta \div 2)$ 
    Incremental linear displacement in the y-axis,  $\Delta y \leftarrow \Delta s \times \sin(\theta + \Delta \theta \div 2)$ 
    Total linear displacement in the x-axis,  $x \leftarrow x + \Delta x$ 
    Total linear displacement in the y-axis,  $y \leftarrow y + \Delta y$ 
end while

```

---

This odometry computed is mathematically correct when the  $\Delta \theta$  tends to zero. In this application the  $\Delta \theta$  tends to zero due to the high update rate of the velocity increments (100 hertz) and a maximum angular velocity of  $\pm 1.0$  radians per second leading to a maximum  $\Delta \theta$  of  $\pm 0.01$  radians.

## Lidar Sensor and the IMU Driver

For the lidar sensor the default ROS 2 Ouster driver [47] was used. This driver teaches the user on how to set up the connection between the 3D lidar sensor and the PC and provides the 3D lidar

and the IMU data on different topics. This driver also has a YAML file for the configuration of the sensor. The following configuration of the lidar sensor driver was used:

**lidar\_mode** It was set to 2048x10. 2048 denotes the highest possible horizontal resolution for a dense lidar scan or a point cloud of the Ouster OS1-128 3D lidar sensor. Horizontal resolution is the number of points in a row of a lidar scan or a point cloud. 10 denotes ten hertz which is the highest possible update rate of the lidar measurements if the horizontal resolution is set to 2048 in this lidar sensor.

**timestamp\_mode** It was set to TIME\_FROM\_ROS\_RECEPTION as all the times used in this application are the ROS times which are synchronized with the system times.

This driver also provides the frame transformations between the sensor frame (`laser_sensor_frame`), the lidar frame (`laser_data_frame`) and the IMU frame (`imu_data_frame`). The frame transformations are provided in the form of a TFMessage [48] on the `/tf_static` topic. A TFMessage contains the following information:

**Timestamp** It is the time at which a TFMessage is published to a topic. It is the ROS time which is synchronized with the system time.

**Parent Frame** It is the frame in reference to which the pose of a child frame is represented.

**Child Frame** It is the frame whose pose is represented in reference to a parent frame.

**Frame Transformation** It is the frame transformation from the parent frame to the child frame in the form of an x, y, z translation and a rotation as a quaternion in x, y, z, w order where w is the real part and x, y, z are the imaginary part of the quaternion.

It should be noted that the frame transformations provided by Ouster in the Ouster software user manual [49] are completely different from the frame transformations provided by this driver. So, if a person is using this driver they should use the frame transformations as mentioned below:

1. The sensor frame in the Ouster software user manual corresponds to the `laser_sensor_frame` in this driver. They are at the same translational positions but they are rotated by 180 degrees about the z-axis with respect to each other.
2. The lidar frame in the Ouster software user manual corresponds to the `laser_data_frame` in this driver. They are at the same translational positions but they are rotated by 180 degrees about the z-axis with respect to each other. They are located at the axis of rotation and the zero degrees beam elevation angle of the lidar sensor.
3. The IMU frame in the Ouster software user manual corresponds to the `imu_data_frame` frame in this driver. They are at the same translational positions but they are rotated by

180 degrees about the z-axis with respect to each other.

More and updated information regarding the frame transformations in this driver can be found in [50].

This driver provides 3D lidar data in the form of a PointCloud2 message [51] on the /points topic at a frequency of about ten hertz. A PointCloud2 message contains the following information:

**Timestamp** It is the time at which a PointCloud2 message is published to a topic. It is the ROS time which is synchronized with the system time.

**Frame** It is the frame in which point cloud data is represented which in this application was the laser\_data\_frame.

**Width** It specifies the number of points in a row of a point cloud which in this application was 2048.

**Height** It specifies the total number of rows in a point cloud which in this application was 128.

**Data** This describes the x, y, z coordinates of the points in a point cloud in a frame described above.

Each PointCloud2 message from this driver contains the full revolution point cloud formed by each 360 degrees rotation of the 3D lidar sensor. This driver also provides the IMU data as an Imu message [52] on the topic /imu at a frequency of about 100 hertz. An Imu message contains the following information:

**Timestamp** It is the time at which an Imu message is published to a topic. It is the ROS time which is synchronized with the system time.

**Frame** It is the frame in which the IMU data is represented, which in this application was the imu\_data\_frame.

**Orientation** It is an orientation estimate as a quaternion provided by an IMU driver. The used IMU driver does not provide an orientation estimate but gives a constant [0, 0, 0, 1] quaternion as the orientation.

**Orientation Covariance** It is the covariance of the orientation estimate from an IMU represented as a 3x3 diagonal matrix with variance about the x, y, z (roll, pitch, yaw) axis in radians squared in row-major order. As the IMU driver used in this application does not provide an orientation estimate, the covariance matrix has all -1.0 in the 3x3 orientation covariance matrix for the localization algorithms to avoid using this orientation estimate.

**Angular Velocity** It is the angular velocity in radians per second about the x, y and z axis from the three-axis gyroscope of an IMU.

**Angular Velocity Covariance** It is the covariance of the angular velocity in radians squared per seconds squared represented as a 3x3 diagonal matrix with variance about the x, y, z axis in row-major order.

**Linear Acceleration** It is the linear acceleration from the three-axis accelerometer in an IMU in meters per second squared about its x, y, z axis. The gravity vector was not removed from the linear accelerations in the used IMU driver. Also when an IMU's axis faces upwards it reports  $+g$  linear acceleration in SI units.

**Linear Acceleration Covariance** It is the covariance of the linear acceleration in  $m^2/s^4$  represented as a 3x3 diagonal matrix with variance in the x, y, z axis in row-major order.

## GNSS Receiver Driver

For the GNSS receiver, the ROS 2 `nmea_navsat_driver` [53] was used which converts the output from the GNSS receiver and publishes them as a `NavSatFix` message [54] on the `/fix` topic. The position output of the GNSS receiver is in the form of NMEA 0183 sentences [55]. The frequency of the position output from the used GNSS receiver was about one hertz. A `NavSatFix` message contains the following information:

**Timestamp** It is the time at which a `NavSatFix` message is published to a topic. It is the ROS time which is synchronized with the system time.

**Status** It describes the status of a `NavSatFix` message as a `NavSatStatus` message [56]. A `NavSatStatus` message contains the fix status type and which GNSS signals were used by a GNSS receiver. For this application, the status was either two or zero, where two denotes ground-based augmentation and zero denotes unaugmented fix. Here the augmentation was provided by Satellitenpositionierungsdienst (SAPOS®) Baden-Württemberg. The status sometimes became zero when the robot was driven near to the buildings. In this application, the GNSS signals were received by the GPS satellites as denoted by one in the `NavSatStatus` message on the `/fix` topic.

**Latitude** It is the latitude of the used GNSS antenna in degrees specified using the WGS 84 system [57].

**Longitude** It is the longitude of the used GNSS antenna in degrees specified using the WGS 84 system.

**Altitude** It is the altitude of the used GNSS antenna in meters specified using the WGS 84 system.

**Position Covariance** It is the covariance of the position of the GNSS receiver in meters squared as a 3x3 diagonal matrix in the local tangent plane coordinates [58] about x, y, z axis in row-major order, where x, y, z axis corresponds to east, north, up coordinates respectively. The variance of the GNSS data is high in the z-axis as the GNSS antenna has satellites above it but not below it for computing its position.

**Position Covariance Type** The position covariance type for the GNSS data in this application was 1 which denotes that the covariance was approximated from the Dilution of Precision.

The parameter useRMC was set to false in this driver as the RMC sentence in the NMEA 0183 protocol does not contain altitude information as mentioned in [59].

## 6.2 Potential Approaches and Problems

### 6.2.1 Point Cloud Map Making and the 3D Localization Challenges Setup

For making the point cloud map and testing the 3D localization challenges, the sensor data has to be provided to the point cloud map making and the 3D localization algorithms. There are two approaches for it which are as follows:

1. Perform the map making and the 3D localization algorithms online which means that the sensor data is provided to them and is also acquired simultaneously. There are many disadvantages of this approach:
  - a) This approach would require high computational power due to the sensor data acquisition and the point cloud map making or the 3D localization algorithms running simultaneously.
  - b) If some mistake is made in the frame transformations, or,
  - c) setting up the algorithms, or,
  - d) if some parameter tuning in the point cloud making or the 3D localization algorithms is needed, or,
  - e) if the results do not look good, or,
  - f) if the same sensor data is needed on multiple point cloud making and the 3D localization algorithms, then exactly the same sensor data needs to be captured by making exactly the same trajectory of the robot. This is practically not possible when the robot is driven by a joystick.

- Record the sensor data while capturing it and then playback it afterwards on the point cloud making and the 3D localization algorithms as many times as a person wants. Through this approach the problems arising from the first approach can be eliminated.

For recording and playing back the sensor data, rosbag2 [60] was used. rosbag2 is used for recording and playing back the messages published on the topics. The CAN frames from the robot's CAN bus, 2D lidar, 3D lidar, IMU, commanded velocity to the robot's tracks, odometry and GNSS receiver data were captured and stored in the bags.

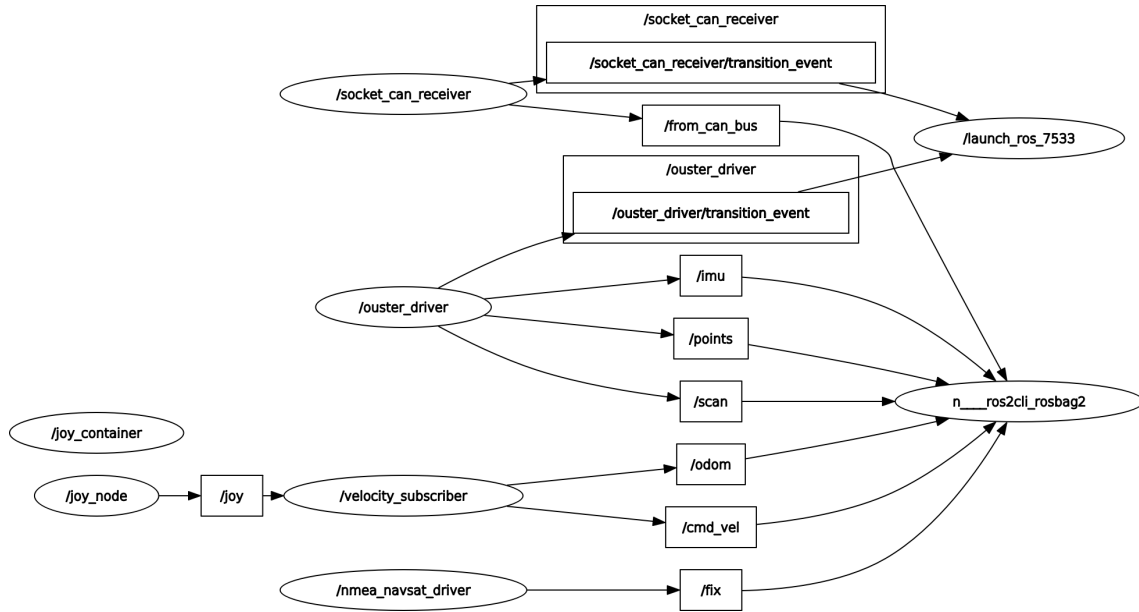


Figure 6.9: rqt\_graph of the Dataset Recording

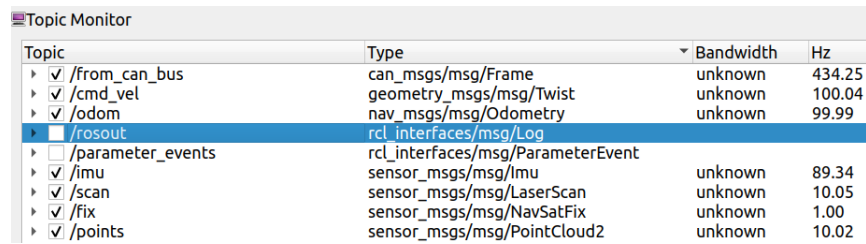
The figure 6.9 shows the rqt\_graph [61] of the dataset's recording. rqt\_graph is a graphical user interface for ROS showing the communication between the nodes and the topics. The ovals represent the nodes and the rectangles represent the topics published or subscribed by them. The explanation of this rqt\_graph is as follows:

- The /joy node publishes the gamepad's commands to the /joy topic.
- The velocity increments sent to the robot is published as a Twist message on the /cmd\_vel topic.
- The odometry computed is published to the /odom topic.
- The CAN frames received from the robot's CAN bus are read by the /socket\_can\_receiver node and published to /from\_can\_bus topic as a CAN Frame message [62]. A CAN Frame message contains the following information:

**Timestamp** It is the time at which a CAN Frame message is published to a topic. It is the ROS time which is synchronized with the system time.

**CAN Frame** It is the eight byte CAN frame.

5. The 2D lidar, the 3D lidar and the IMU data from the /ouster\_driver node is published to /scan, /points and /imu topics respectively.
6. The GNSS data is published to the /fix topic.
7. All the data in the topics is stored in the bag.



| Topic                                             | Type                              | Bandwidth | Hz     |
|---------------------------------------------------|-----------------------------------|-----------|--------|
| <input checked="" type="checkbox"/> /from_can_bus | can_msgs/msg/Frame                | unknown   | 434.25 |
| <input checked="" type="checkbox"/> /cmd_vel      | geometry_msgs/msg/Twist           | unknown   | 100.04 |
| <input checked="" type="checkbox"/> /odom         | nav_msgs/msg/Odometry             | unknown   | 99.99  |
| <input checked="" type="checkbox"/> /rosout       | rcl_interfaces/msg/Log            |           |        |
| <input type="checkbox"/> /parameter_events        | rcl_interfaces/msg/ParameterEvent |           |        |
| <input checked="" type="checkbox"/> /imu          | sensor_msgs/msg/Imu               | unknown   | 89.34  |
| <input checked="" type="checkbox"/> /scan         | sensor_msgs/msg/LaserScan         | unknown   | 10.05  |
| <input checked="" type="checkbox"/> /fix          | sensor_msgs/msg/NavSatFix         | unknown   | 1.00   |
| <input checked="" type="checkbox"/> /points       | sensor_msgs/msg/PointCloud2       | unknown   | 10.02  |

Figure 6.10: Frequencies of the Sensor Data as Recorded in a Bag at an Instant

The frequencies of the sensor data as recorded in a bag at an instant is shown in 6.10. The tool used for monitoring the data frequency in the topics is called the rqt\_topic [63].

The sensor data for making the point cloud map and the 3D localization challenges were recorded in a single day on a weekend. This is because the sensor data was recorded in the InMach's parking lot and if it was recorded on different days then the environment would not be static. There could be more or less number of cars in the parking lot depending on the time of day which would defeat the aim of comparing the quality of the 3D localization algorithms on the same environment and the point cloud map. Moreover the GNSS data accuracy depends upon the visibility of the satellites and the weather which could be different each time of day.

All the sensor data recording for making the point cloud map and the 3D localization challenges was made on 25<sup>th</sup> September 2021 on a clear sky starting from about 15:30 in the afternoon till about 18:30 in the evening. For recording the sensor data for all the 3D localization challenges except in the fog challenge, a loop was made by driving the robot starting from the origin of the map and finally coming back to its origin. This was to check if or how much the position of the robot is deviated from its starting position when it finally comes back to its starting position, i.e., the origin of the map. An obstacle arena was defined in the map for putting the obstacles. In all the 3D localization challenges the robot was driven at a maximum of  $\pm 0.5$  meters per second linear velocity and  $\pm 0.5$  radians per second angular velocity unless otherwise mentioned. The sensor data is recorded for the following 3D localization challenges:

**No obstacles** Drive in the map with no obstacles in the obstacle arena. This was to check that



how well are the 3D localization algorithms performing when no obstacles are there in the map and the robot travels a smooth and a slow path like a walking human.

**High velocity** Drive in the map with no obstacles in the obstacle arena with a maximum of  $\pm 1.0$  meters per second linear velocity and  $\pm 1.0$  radians per second angular velocity of the robot. The reasons for choosing this 3D localization challenge were:

1. This was to check how well are the 3D localization algorithms performing when the robot travels at high velocities and accelerations which corrupts the linear acceleration measurements of the three-axis accelerometer in the IMU for the reasons discussed in 6.1 and 6.1.1. This would also be a good check of how accurately the 3D localization algorithms track the orientation of the robot with the accelerometer and gyroscope data from the IMU which is discussed in 6.2.4.
2. In this 3D localization challenge the robot gets a lidar scan every ten centimeters it moves, whereas in the no obstacles 3D localization challenge it gets a lidar scan every five centimeters it moves. Therefore the 3D localization algorithms need to rely for more time on its motion model till it gets a lidar scan to determine the pose of the robot in the map. This would be a good check of how accurate is the odometry data, IMU data and the motion model.
3. Also, as the robot moves faster, each one full revolution point cloud would be more distorted as the robot now moves a distance of ten centimeters during the capturing of each one full revolution point cloud. This would be a good check of how well the 3D localization algorithms handle the point cloud undistortion and do the scan-matching.

**Fog** Drive forward for some meters in the map and then stop where the fog machine is producing the fog around the lidar sensor. This was to check how well are the 3D localization algorithms performing when the lidar data is corrupted by the fog around it.

**Pitch** Drive in the map with both the robot's tracks on the slope in the obstacle arena. The slope dimensions are 3.19 m x 1.19 m x 0.2 m and its picture is shown in 6.11. The maximum pitch angle was about seven degrees. The reasons for choosing this 3D localization challenge were:

1. In a real outdoor environment the ground is not flat, but is uneven and has slopes. Therefore this 3D localization challenge was chosen to simulate the real outdoor environment.
2. It was also chosen to check how accurately are the 3D localization algorithms are performing when they do not see much point cloud data on one side of the robot for

scan-matching and have to rely on the odometry and the IMU data to determine the robot's pose with respect to the map.

**Roll** Drive in the map with the right track on the slope and the left track on the ground in the obstacle arena. The slope used for this 3D localization challenge was the same as that of the pitch 3D localization challenge. The maximum roll angle is about twelve degrees. The robot went over the slope while returning back to the origin of the map after taking a U-turn at the end of the map in both the pitch and the roll 3D localization challenge. This 3D localization challenge was chosen for the same reasons as mentioned above in the pitch 3D localization challenge.



Figure 6.11: Slope Used for the Pitch and the Roll 3D Localization Challenge

## 6.2.2 rosbag2 Dropping Messages

By looking at the metadata.yaml file of the bags recorded, it was observed that the bags are dropping a lot of messages. For example, the frequency of the lidar data is ten hertz and the IMU data is 100 hertz but the frequency of the lidar and the IMU data recorded in the bags were just 4.5 hertz. So the rosbag2 foxy-future branch [64] was used. It is a back port of its ROS 2 Galactic Geochelone version which has performance improvements and new features over its predecessor, i.e., the ROS 2 Foxy Fitzroy version, which was used in this application.

## 6.2.3 Frame Transformations in ROS

In ROS, all the frames are right handed, and the measurements are expressed in SI units as mentioned in [65]. Also, each frame can only have one parent frame. In ROS, the moving frames are published to the /tf topic, and the static frames to the /tf\_static topic both in the form of a TFMessage. The frames used in this application for the robot, the sensors and the 3D localization algorithms are as follows:

- enu** This is the ENU (east, north, up) frame in which the ground truth RTK GNSS data is converted to compare with the 3D positions provided by the 3D localization algorithms in the map frame. It is also called as the local Cartesian coordinate frame and is a static frame.
- map** This is the static local map frame relative to which the localization algorithms track the pose of the robot. Its z-axis points upwards, and its -z-axis is aligned with the direction of the gravity vector due to the presence of a three-axis accelerometer in the IMU.

**odom** This is the parent frame relative to which the odometry of the robot is represented in the base\_link frame. It drifts away with time due to the errors in the odometry discussed in 6.2.11, and therefore is non-static. When the initial pose of the robot is not estimated by a localization algorithm it is coincident with the map frame in which the pose of the robot is represented by a localization algorithm until it drifts away with time. The work of a localization algorithm is in fact to calculate this drift and therefore publish the map to odom frame transformation as mentioned in [9]. If somebody has a perfect ground truth odometry using RTK GNSS receivers as discussed in 6.2.8, which does not drift away with time then this frame will be the same as the map frame in which the pose of the robot is represented by the localization algorithms. If this is the case, then there would be no requirement of a localization algorithm or any other sensors for localization as the odometry is perfect.

**base\_link** This is the frame representing the base of the robot. Its origin is defined at the center of the length and width of the two caterpillar tracks of the robot. Due to the suspension system of the robot its origin is vertically defined at the base of the metallic body of the robot so that the 3D lidar sensor and the GNSS antenna is static with respect to this frame. Its x-axis points in the forward moving direction direction of the robot and is parallel to the tracks of the robot.

**laser\_sensor\_frame** It is the 3D lidar sensor frame representing the pose of the 3D lidar sensor. It is defined as a static frame transformation with respect to the base\_link frame with a rotation of 180 degrees about the z-axis due to the reasons mentioned in 6.1.2.

**laser\_data\_frame** It is the frame in which the 3D lidar data is expressed. It is defined as a static frame transformation with respect to the laser\_sensor\_frame.

**imu\_data\_frame** It is the frame in which the IMU data is expressed. It is defined as a static frame transformation with respect to the laser\_sensor\_frame.

**gps** It is defined at the position of the L1 (single-band) GPS signal receiver, and its x-axis is parallel to and points in the direction of the x-axis of the base\_link of the robot. The 3D localization algorithms' poses are tracked as a frame transformation from the map to the gps frame.

**ground\_truth** It is defined at the position of the L1 (single-band) GPS signal receiver and this is the frame whose positions are recorded as GNSS data in the sensor data bags. These GNSS positions are converted to represent GNSS data in the enu frame for comparison with the positions of the robot provided by the 3D localization algorithms.

The frame transformations tree used in this application is shown in 6.12.

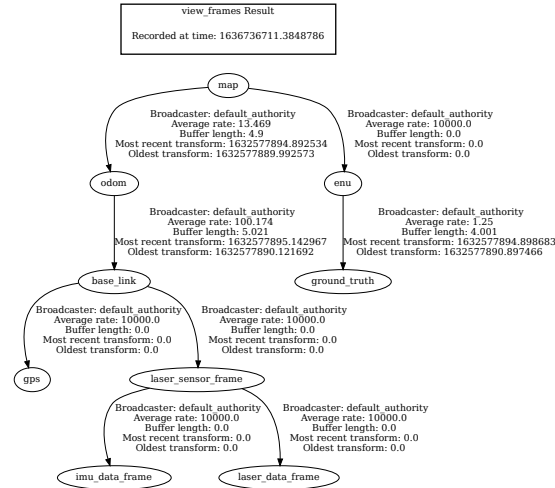


Figure 6.12: Frame Transformations Tree

## 6.2.4 Point Cloud Map Making

For making a map of the environment there are two approaches:

**Mapping** In this approach the ground truth poses of the robot are already known through methods discussed in 6.2.8 and only scan-matching is performed for making the map.

**SLAM (simultaneous localization and mapping)** In this approach, both localization and mapping is performed simultaneously for making the map. In this master thesis, the SLAM approach is used for making the point cloud map as a very accurate ground truth was not available as discussed in 7.1.

For making the point cloud map, the Google Cartographer 3D SLAM algorithm [15] was used. It can make maps with five centimeters resolution and also solves the loop closure problem. Its working is as follows:

1. The input lidar point clouds are downsampled using a voxel filter. The voxel filter is used to downsample the input point clouds using cubes of constant size, and only keeping the centroid of those points in the cubes. In this 3D SLAM algorithm, the voxel filter size chosen is a cube of edge length fifteen centimeters, which is also its default. The filtering of the input point clouds are performed, as the far away objects have less points than the nearby points. This is because of the same horizontal and vertical resolution of the lidar for nearby and far away points, which decreases the number of laser hits as the distance to the object increases. The filtering is also performed to decrease the computational complexity for the scan-matching, caused due to large number of points in the input point cloud. The raw input point cloud in this application has 262,144 points per one full revolution point cloud.

The raw input point cloud and its voxelized version is shown in 6.13 and 6.14. Here the raw input point cloud has 262,144 points. After applying the voxel filter to it, with a cube of edge length one meter it is downsampled to about 4,000 points.

2. This SLAM algorithm has two components, called the local and the global SLAM. The work of the local slam is to build submaps and the work of the global slam is to tie them together as mentioned in [22]. The lidar scans are inserted in the submaps using scan-matching and the initial guess of its pose using the pose extrapolator component. The scan-matching component used in this SLAM algorithm is the CeresScanMatcher which uses the Ceres Solver [66]. The Ceres Solver is used to solve the non-linear least squares problem. It is used in this SLAM algorithm to find the frame transformation between the lidar scans in the submap (reference scan) and the current scans that minimizes the least squared error between them. If the motion (frame transformation) between the reference and the current scans is significant enough as specified in its motion filter component [67], then the current scans are inserted in the submap. The pose extrapolator component is used to predict the pose of the robot using the odometry and the IMU data.
3. The global SLAM is based on the pose graph optimization also called the GraphSLAM algorithm. In the GraphSLAM algorithm the robot's poses are linked through motion arcs and the robot's poses are linked to the map features using measurement arcs. The robot's poses are estimated through the odometry and the IMU data and the map features are estimated with the lidar data. These links are called edges in the graph built by the GraphSLAM algorithm. Each edge is a constraint and sum of all these constraints results in a least squares problem. Minimizing the sum of these least squares, result in the most likely map and the most likely robot trajectory as mentioned in [9]. The global SLAM ties the submaps formed by the local SLAM together and also finds the loop closures. When the robot revisits it previously mapped area, it observes the same features as observed earlier when it mapped them. So by the process of scan-matching, the scans in the earlier mapped submaps are matched with the current scans. If they match, then the loop is closed which is called loop closure. If a loop closure is found, then the scans and the robot trajectory in the submaps is corrected accordingly. This also corrects for the drift in the local SLAM caused over time.

For making the point cloud map, the 3D lidar, the IMU and the odometry is used. This 3D SLAM algorithm requires an IMU for determining the direction of gravity using its imu\_tracker component [68]. This algorithm requires the angular velocity about x, y, z axis and the linear acceleration in x, y, z axis from an IMU but does not require an orientation estimate from it. It keeps track of the orientations also from this imu\_tracker. Therefore the used IMU driver was compatible with this 3D SLAM algorithm as it provides the angular velocity about x, y, z axis and linear acceleration in x, y, z axis but does not provide an orientation estimate from the IMU. The tuning parameters of this algorithm have to be defined in a Lua configuration file [69]. For using the Ouster OS1-128 3D lidar sensor with this 3D SLAM algorithm, the parameter TRAJECTORY\_BUILDER\_3D.num\_accumulated\_range\_data should be set to one due to the reason men-

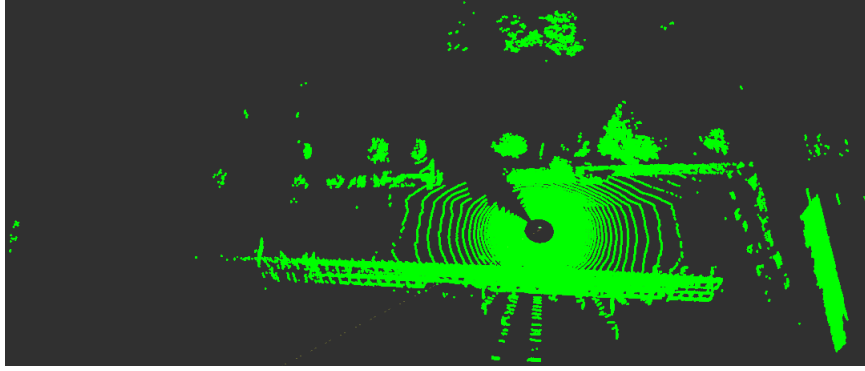


Figure 6.13: Raw Input Point Cloud

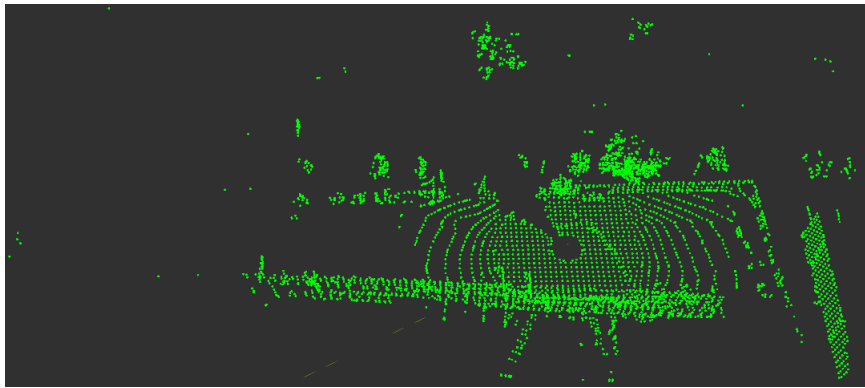


Figure 6.14: Voxelized Input Point Cloud

tioned in 6.1.2. The minimum and maximum range of the input 3D lidar point cloud data was kept the same as the default from one to 60 meters respectively. The reasons for the range filtering of the input point cloud data are mentioned in 6.2.11.

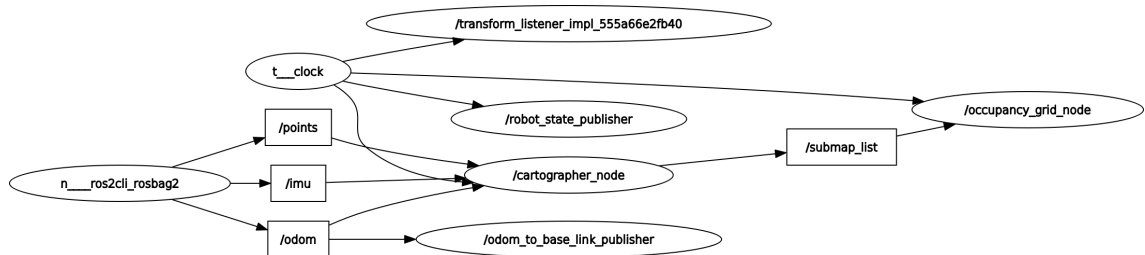


Figure 6.15: rqt\_graph of the Google Cartographer 3D SLAM Algorithm

The rqt\_graph of this SLAM algorithm is shown in 6.15. Its explanation is as follows:

1. The 3D lidar, IMU and the odometry data is played from the sensor data bag through the /points, /imu and the /odom topic respectively.
2. The /robot\_state\_publisher node publishes the static frame transformations from the UR-

DF file of the robot model. More information about the URDF file and the robot\_state\_publisher package is given in 6.2.10.

3. The /cartographer\_node node builds the optimized submaps and publishes them to the /submap\_list topic.
4. The /submap\_list topic is subscribed by the occupancy\_grid\_node to build a 2D occupancy grid map of the environment. The occupancy grid map formed by this SLAM algorithm is shown in 6.18.

For recording the sensor data for making the point cloud map, the following procedure was followed:

1. The robot was driven starting from the origin of the map to the end point of the map, at a maximum and almost constant 0.2 meters per second linear velocity and 0.2 radians per second angular velocity of the robot.
2. A loop by driving the robot starting from the origin of the map and finally coming back to the origin of the map was not made.
3. A constant and a very low robot velocity was chosen as when the robot was driven at high velocities with high accelerations or decelerations the point cloud map made by this 3D SLAM algorithm was not accurate. This was because the three-axis accelerometer in the IMU experienced vibrations when the robot was driven at high velocities with high accelerations or decelerations. This was due to the following reasons:
  - a) When the robot traveled on uneven grounds with high velocities, the three-axis accelerometer in the IMU experienced vertical oscillations because of the suspension system of the robot. These vertical oscillations caused vertical linear acceleration measurements (in addition to the acceleration due to gravity) in the z-axis of the IMU.
  - b) When the robot is traveling with high accelerations or the decelerations, the three-axis accelerometer in the IMU experiences horizontal oscillations as it was mounted on the aluminum frame on the robot. These horizontal oscillations causes horizontal linear acceleration measurements in the x-axis and the y-axis of the IMU. If there are some errors in these linear acceleration measurements, then it could degrade the quality of the point cloud undistortion and also the alignment of the -z-axis of the map with the gravity vector as discussed in 6.1.

### 6.2.5 2D Occupancy Grid Map

In a 2D occupancy grid map, the map is a gray scale image, usually as a PGM file divided into pixels or grids of constant size. Its meta-data contains the following information:

**image** It is the name of the map image file.

**resolution** In this application each grid has a size of five centimeters by five centimeters, which is also called the resolution of the map.

**origin** The origin of the map is the 2D pose of the lower left grid of the map.

**occupied\_thresh** If a grid is black, it is given the value one, which means it is occupied, and if it is white it is given the value zero, which means it is not occupied or it is free. Each grid color is from zero and 255 where zero denotes black and 255 denotes white. All these gray scale values between zero and 255 are normalized from zero to one to assign a probability value to each grid. If the probability value is above a given occupied threshold value, then that grid is considered occupied, i.e., assigned a value one.

**free\_thresh** If the probability value is below the given free threshold value, it is considered as free, i.e., assigned a value zero. If the probability value is above the free threshold value and below the occupied threshold value it is assigned a value of -1, showing that its occupancy is unknown.

### 6.2.6 Point Cloud Map PCD File Output from the Google Cartographer 3D SLAM Algorithm

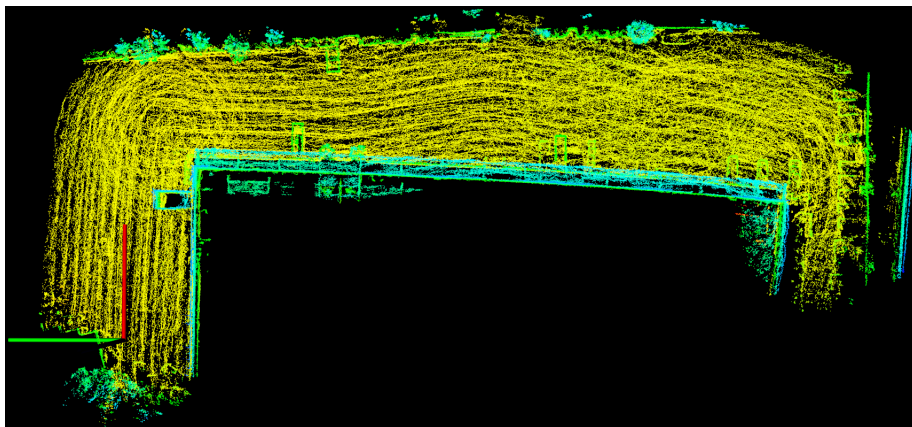


Figure 6.16: Point Cloud Map Made by the Google Cartographer 3D SLAM Algorithm

The Google Cartographer 3D SLAM algorithm stores the point cloud map and the 3D SLAM trajectory in a .pbstream file. For its 3D localization mode on this point cloud map the .pbstream file



output of its 3D SLAM algorithm has to be provided. Therefore, for its 3D localization algorithm, a point cloud map is not needed in the form of a PCD file. The contents of a PCD file are described in 6.2.7. But the other 3D lidar-based localization algorithms require a map in the form of a PCD file for the 3D localization.

This 3D SLAM algorithm in ROS 1 has a component called the `cartographer_assets_writer` [22] which is used to output the point cloud map in the form of a PCD file made by its 3D SLAM algorithm. This component replays the same sensor data bag on the 3D SLAM trajectory in the `.pbstream` file which is earlier generated by its 3D SLAM algorithm using that sensor data bag. This process is similar to the mapping process. But its 3D localization algorithm uses the point cloud map stored in the `.pbstream` file for the 3D localization. Therefore if a person is using its `assets writer` component for providing the point cloud map to the other 3D lidar-based localization algorithms, it would be unfair. This is because the trajectory output in the `.pbstream` file generated by its 3D SLAM algorithm would be better as it undergoes pose graph optimization leading to better point cloud maps as discussed in 6.2.4. The `assets writer` component was also not yet available in ROS 2 Foxy Fitzroy as mentioned in [70]. Therefore by taking help from [71], C++ code was written to extract the point cloud map as a PCD file from the `.pbstream` file generated by this 3D SLAM algorithm using the Point Cloud Library (PCL) [72].

This 3D SLAM algorithm considers the IMU frame as the tracking frame. The tracking frame is the frame in which the sensor data (3D lidar, IMU and the odometry) it is expressed and is aligned with the direction of the gravity vector which means that the gravity vector points in its  $-z$  axis direction. It is not fixed and changes over time with different trajectories as mentioned in [73] and [22]. Therefore the origin of this point cloud map is also the starting pose of the IMU frame when this 3D SLAM algorithm is started.

## 6.2.7 PCD File

A PCD file contains the following information about a point cloud:

**Width** It is the width of a point cloud. More information about the width of a point cloud is discussed in 6.1.2.

**Height** It is the height of a point cloud. More information about the height of a point cloud is discussed in 6.1.2.

**Points** The total number of points in a point cloud, which is equal to its width multiplied by its height.

**Data** An array of vector of points usually containing  $x$ ,  $y$  and  $z$  coordinates of the points in a point cloud.

### 6.2.8 Comparison of the Quality of the 3D Localization Algorithms with the Ground Truth RTK GNSS Receiver Positions

Only one RTK GNSS receiver and antenna were placed on the robot, which can only provide the positions of the RTK GNSS receiver but no orientations. For comparing the 6D pose provided by the 3D localization algorithms with the ground truth RTK GNSS receiver positions the following techniques can be used:

- Fusion of the positions from three RTK GNSS receivers and antennas to compute the 6D pose of the robot.
- Fusion of the RTK GNSS receiver positions and the orientation estimates from an IMU to compute the 6D pose of the robot.

But these techniques were out of scope of this master thesis. Further arguments to compare only the 3D positions of the robot provided by the 3D localization algorithms with the ground truth RTK GNSS receiver positions are:

1. If the 3D positions provided by the 3D localization algorithms are accurate then the orientations provided by them would also be accurate as the robot is driven almost linearly in the map.
2. The robot is also not like a drone which can have quite significant roll, pitch and yaw angles.
3. As the robot is a differential tracked-wheel drive robot, it has non-holonomic constraints which do not allow it to move sideways.

Therefore in this master thesis, only the 3D positions of the robot provided by the 3D localization algorithms are compared with the ground truth RTK GNSS receiver positions.

### 6.2.9 GNSS Data Coordinate Frames

The ground truth RTK GNSS positions recorded in the sensor data bags were in the form of WGS 84 latitude, longitude and altitude coordinates. But the 3D localization algorithms provide the position of the robot in x, y, z coordinates in the map frame. For comparison of the positions provided by the 3D localization algorithms and the ground truth RTK GNSS positions, a common coordinate system is required. Therefore the common coordinate frame chosen is the local Cartesian coordinate frame (ENU frame) [74]. For the conversion of WGS 84 latitude, longitude and altitude coordinates (LLA coordinates) to the local Cartesian coordinates, GeographicLib [75] was used. GeographicLib contains a C++ class named LocalCartesian to convert between LLA and

ENU coordinates. The LLA coordinates of the origin were provided to the constructor of the LocalCartesian class. The LLA coordinates of the origin were the first LLA coordinates recorded in the sensor data bag used for making the map by the Google Cartographer 3D SLAM algorithm. To convert the LLA coordinates to the ENU coordinates the Forward function of the LocalCartesian was used. It is used to compute the ENU coordinates of the input LLA coordinates. The ENU frame's x-axis points due east, y-axis point due north and the z-axis points up and is normal to the earth's ellipsoid. The point cloud map created by Google Cartographer 3D SLAM algorithm has its origin at the starting pose of the IMU frame as discussed in 6.2.6. But that map's y-axis does not point due North. So for a common coordinate system, the local Cartesian coordinate frame needs to be rotated about its z-axis to align it with the map frame. The process of this alignment was as follows:

1. The direction of the True North (y-axis of the local Cartesian coordinate frame) with respect to the y-axis of the map frame was determined using the ETRS 89/UTM ZONE 32N [76] coordinates of the InMach's office. It was determined by finding the inverse tangent of the angle between the two corner points of the InMach's office from the vertical direction which point due True North.
2. The alignment was also verified by using the GIMP software [77] for aligning the image of the real map from the Google Maps [78] and the 2D occupancy grid map output from the Google Cartographer pbstream\_to\_ros\_map component [79] one over the other and finding the angle between them. The Google Cartographer pbstream\_to\_ros\_map component outputs the 2D projection of the 3D point cloud map stored in a .pbstream file. The real map image from the Google Maps is shown in 6.17 and the 2D occupancy grid map output from the Google Cartographer pbstream\_to\_ros\_map component is shown in 6.18.
3. The alignment was further corrected by using the evo [80] implementation of the Umeyama alignment [81]. The Umeyama alignment is from Shinji Umeyama and is used to find the frame transformation ( $3 \times 3$  rotation matrix and x, y, z translation) between two sets of points which minimizes the least mean squared error between them. Evo is a Python package from Michael Grupp used for handling, evaluating and comparing the trajectory output of the odometry and the SLAM algorithms. It was also used for the evaluation and comparison of the trajectories of the robot made by the 3D localization algorithms with the ground truth RTK GNSS data in this master thesis. The file format used for the trajectory outputs was the TUM (Technical University of Munich) dataset format [82] which contains the timestamps and the 6D poses of an object. Only the positions of the robot provided by the 3D localization algorithms were logged ignoring the orientations due to the reasons discussed in 6.2.8. The positions of the robot provided by the 3D localization algorithms and the ground truth RTK GNSS data were in the form of x, y, z coordinates. The procedure for this alignment was as follows:
  - a) The 3D SLAM trajectory from the .pbstream file output of the Google Cartographer 3D SLAM algorithm was extracted by writing the C++ code using [83]. It was extracted in

the form of timestamps and  $x, y, z$  positions, ignoring the orientations as discussed in 6.2.8. These positions were the positions of the IMU frame with respect to the map frame.

- b) The ground truth RTK GNSS positions of the IMU frame were extracted in the ENU frame from the sensor data bag used for making the point cloud map.
- c) Then these two trajectories were aligned using Umeyama alignment implemented in evo. These two trajectories were aligned till the point the map was accurate, which is further discussed in 7.1.

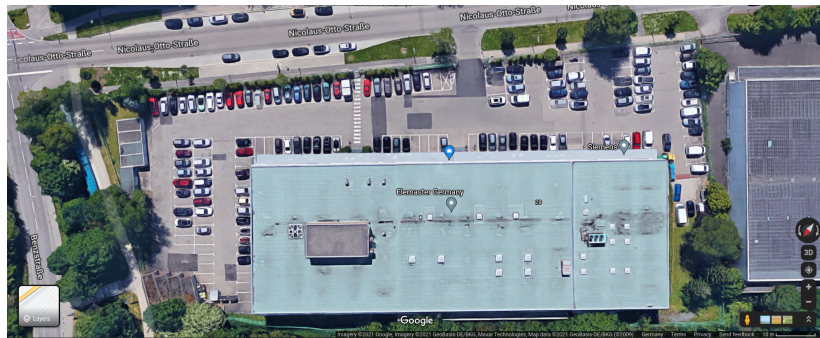


Figure 6.17: Real Map Image from Google Maps

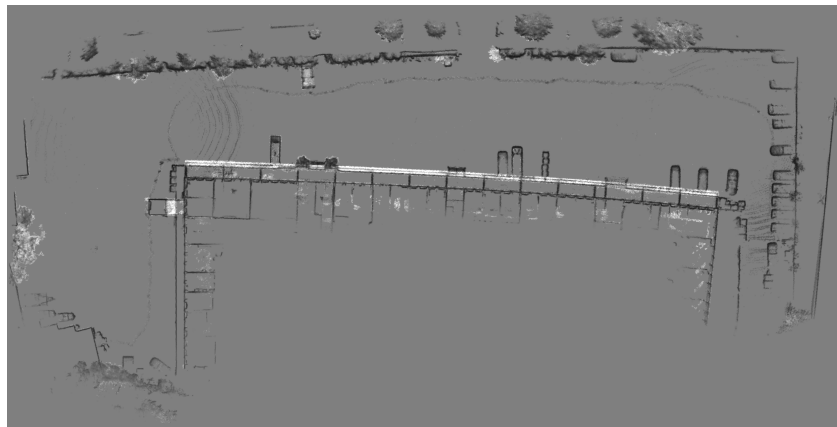


Figure 6.18: 2D Occupancy Grid Map Output from the Google Cartographer pbstream\_to\_ros\_map Component

### 6.2.10 Visualization of the Robot Model in RViz

In ROS, RViz [84] is used to visualize the sensor data, frames and the maps. For visualizing the robot model in RViz, the following procedure was followed:

1. The 3D CAD (computer-aided design) STEP (Standard for the Exchange of Product model data) files [85] of the robot and the sensors were converted to STL (STereoLithography) file format [86] using FreeCAD [87].
2. Then the STL files were converted to COLLADA (COLLABorative Design Activity) file format [88] using Blender [89] for color as STL files do not contain color information, as discussed in [90].
3. In ROS, a URDF (Unified Robot Description Format) file [91] is made in the XML (Extensible Markup Language) format [92] to represent a robot model. It contains the links which represent a robot's visual model and the joints which represent the frame transformations between those links. The COLLADA files were imported in the URDF file of the robot model as URDF supports COLLADA files, as mentioned in [93].
4. Then the robot\_state\_publisher package [94] is used to publish the static frame transformations in the URDF file of the robot model as a TFmessage on the /tf\_static topic at a default frequency of twenty hertz. The URDF file of the robot model is imported in the robot\_state\_publisher package as mentioned in [14].

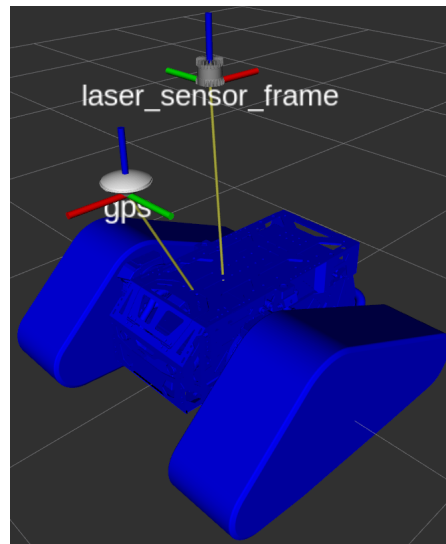


Figure 6.19: Visualization of the Robot Model along with its Frames in RViz

The visualization of the robot model along with its frames in RViz is shown in 6.19.

### 6.2.11 3D Localization Algorithms Discussed, Evaluated and Compared

In this master thesis, the following three 3D localization algorithms were evaluated and compared:

**Google Cartographer 3D Localization Algorithm** It is a component of the Google Cartographer 3D SLAM algorithm used to localize on the .pbstream output of its 3D SLAM algorithm by the process of scan-matching, as discussed in 6.2.6. Its working is same as its 3D SLAM algorithm, but instead of matching the scans with the scans in the submap, the scans are matched with the final optimized point cloud map formed by its 3D SLAM algorithm. The sensors used for this 3D localization algorithm are the same as of its 3D SLAM algorithm, i.e., the 3D lidar, the IMU and the odometry. All the parameters for this 3D localization algorithm were kept the same as of its 3D SLAM algorithm, except the following:

1. For the 3D localization mode of this 3D SLAM algorithm the parameter TRAJECTORY\_BUILDER.pure\_localization should be set to true in the Lua configuration file.
2. The parameter POSE\_GRAPH.optimize\_every\_n\_nodes was set to zero as the pose graph optimization for aligning the submaps and checking for the loop closures is not required for the 3D localization mode of this 3D SLAM algorithm.

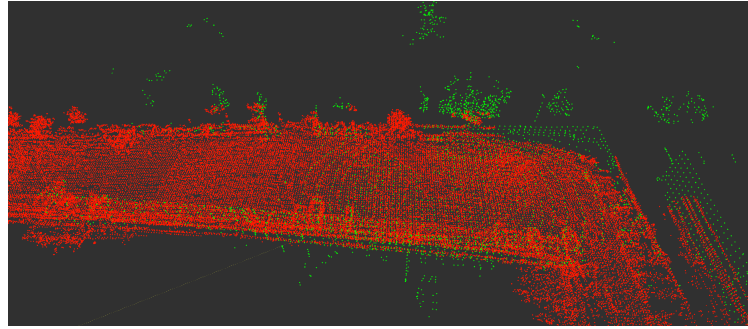


Figure 6.20: 3D NDT Scan-Matching in the Autware.Auto 3D NDT Localization Algorithm

**Autware.Auto 3D NDT Localization Algorithm** It is based on the 3D NDT scan-matching as discussed in 5. The sensors used for this 3D localization algorithm are the 3D lidar and the odometry. It does not use (or require) an IMU as a sensor input for its 3D localization algorithm. A screenshot showing the 3D NDT scan-matching in this 3D localization algorithm is shown in 6.20. The point cloud map is shown in red color and the filtered and the downsampled input point cloud is shown in green color.

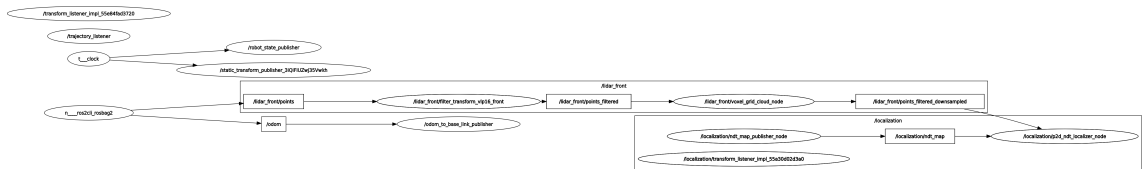


Figure 6.21: rqt\_graph of the Autware.Auto 3D NDT Localization Algorithm

The rqt\_graph of this 3D localization algorithm is shown in 6.21. Its explanation is as follows:

1. The 3D lidar and the odometry data is played from the sensor data bag through the /lidar\_front/points and the /odom topics respectively.
2. The /robot\_state\_publisher node publishes the static frame transformations from the URDF file of the robot model.
3. The /lidar\_front/filter\_transform\_vlp\_16\_front node publishes the 3D lidar point cloud data on the topic /lidar\_front\_points\_filtered after performing the following operations to the input point cloud data:

**Range Filtering** This component filters the input point cloud data between a minimum and a maximum range of 1.5 meters and 150 meters respectively. The minimum range filtering on the input point cloud data is performed so that the robot on which the 3D lidar sensor is mounted does not itself become a part of it. The maximum range filtering on the input cloud data is performed because the points very far away from the lidar sensor can come from reflections. Also when the input point cloud is being undistorted with the IMU data, then if the IMU data is noisy or is providing inaccurate values, then even a small error in the orientation estimates of the point cloud can create large errors in the positions of the far away points in the input point cloud.

**Angle Filtering** This component is used to filter the input point cloud data between an angle range. This is performed as some of the lidar sensors are mounted on the robot such that a part of their 360 degrees horizontal field of view is blocked by the robot itself.

**Static Frame Transformation** This component applies a static frame transformation to the input point cloud data to transform it from the laser\_data\_frame to the base\_link frame. More about these frames is discussed in 6.2.3.

4. The /lidar\_front/voxel\_grid\_cloud\_node node downsamples the filtered and transformed input point cloud data coming from the /lidar\_front\_points\_filtered topic using a voxel filter with a cube of edge length one meter. It then publishes the resulting point cloud data on the /lidar\_front/points\_filtered\_downsampled topic. More about the voxel filter is discussed in 6.2.4.
5. The /localization\_ndt\_map\_publisher node reads the PCD file of the input point cloud map data and publishes the point cloud map data as a PointCloud2 message after voxelizing it with a cube of edge length 3.5 meters on the /localization/ndt\_map topic.
6. The voxelized input point cloud map and the filtered, voxelized and the transformed input point cloud data published on the /localization/ndt\_map and the lidar\_front/points\_filtered\_downsampled topics respectively, are subscribed by the /localiza-

tion/p2d\_ndt\_localizer\_node. This node then performs the 3D localization algorithm with the initial estimate of the pose of the robot with respect to the map provided in the YAML parameter file.

```
root@5751d3d19168:/# ros2 node info /pcl_localization
/pcl_localization
Subscribers:
  /initialpose: geometry_msgs/msg/PoseStamped
  /map: sensor_msgs/msg/PointCloud2
  /odom: nav_msgs/msg/Odometry
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /points: sensor_msgs/msg/PointCloud2
Publishers:
  /initial_map: sensor_msgs/msg/PointCloud2
  /parameter_events: rcl_interfaces/msg/ParameterEvent
  /path: nav_msgs/msg/Path
  /pcl_localization/transition_event: lifecycle_msgs/msg/TransitionEvent
  /pcl_pose: geometry_msgs/msg/PoseStamped
  /rosout: rcl_interfaces/msg/Log
  /tf: tf2_msgs/msg/TFMessage
```

Figure 6.22: pcl\_localization\_ros2 3D Localization Algorithm Node Showing its Published and Subscribed Topics

**pcl\_localization\_ros2 3D PCL NDT Localization Algorithm** It is a 3D lidar-based localization algorithm using the NDT algorithm in the Point Cloud Library (PCL). The sensor used for this 3D localization algorithm is the 3D lidar only. This is because of the following reasons:

1. The IMU required for this 3D localization algorithm is a nine-axis IMU which includes a three-axis gyroscope, three-axis accelerometer and a three-axis magnetometer and which also provides an orientation estimate. But the IMU used in this application is a six-axis IMU which only has three-axis gyroscope and three-axis accelerometer, providing angular velocity, and linear acceleration but no orientation estimate as discussed in 6.1.2.
2. When the odometry was used as an input to this 3D localization algorithm then the robot's poses were not accurate as they were going outside the map. This was because of this 3D localization algorithm believing the odometry to be accurate which actually is not as it drifts away. This is because of the following reasons:
  - a) The robot used is a tracked wheel robot but it is modeled as a differential wheeled drive robot. Therefore the wheelbase is assumed to be constant. The wheelbase used for computing the yaw rates is chosen as the distance between the center of the two tracks assuming that the robot turns about the center of each track. But actually the point of rotation of the robot about the two tracks could be anywhere on the tracks leading to errors when computing the yaw angles from the yaw rates.
  - b) Also as the ground on which the robot is driven is also not flat and the odometry is modeled in 2D, considering the ground as flat. Therefore the odometry only has x, y and yaw component but no z, roll and pitch which leads to errors when the robot drives on an uneven ground.



The `/pcl_localization` node performs the localization. The topics subscribed and published by it are shown in 6.22. The explanation of the used topics in this node is as follows:

1. This node reads the PCD file of the input point cloud map data. This input point cloud map is voxelized into grids of cubes of edge length three meters which is called the NDT resolution.
2. This node subscribes to the input 3D lidar point cloud data published through the `/points` topic. The input point cloud is range filtered from one meter to 100 meters and is voxelized into cubes of edge length one meter.
3. This node then performs the 3D localization algorithm with the initial estimate of the pose of the robot with respect to the point cloud map provided in the YAML parameter file.
4. This 3D localization algorithm publishes the 6D pose of the robot with respect to the map frame on the `/tf` topic.

# 7 Results

## 7.1 Google Cartographer 3D SLAM Quality

The evaluation and the comparison of the 3D localization algorithms' trajectories with the ground truth RTK GNSS data is valid and fair only if the point cloud map made by the Google Cartographer 3D SLAM algorithm was accurate. The Google Cartographer SLAM algorithm and the ground truth RTK GNSS data would be called as Cartographer SLAM algorithm and the ground truth data respectively, further in this master thesis. Therefore, first the quality of the Cartographer 3D SLAM needs to be evaluated and compared with the ground truth data before commenting on the quality of the 3D localization algorithms. The SLAM has two components - mapping and localization.

The quality of the mapping by the Cartographer 3D SLAM algorithm is evaluated and compared by visualizing the point cloud map output from it and its 2D projection, and comparing it with the ground truth map from the Google Maps. It is clearly visible in 6.18 and 6.16, that the long wall of the building in the map made by this 3D SLAM algorithm is curved with respect to the ground truth map from the Google Maps, shown in 6.17, which shows that the quality of this map deteriorates far away from its origin. The map is accurate when the robot is driven straight in the course. It starts deteriorating when the robot takes a right turn.

### 7.1.1 Trajectories Comparisons

The quality of the 3D localization in the Cartographer 3D SLAM is evaluated and compared by plotting its 3D SLAM trajectory with the ground truth data in 7.1. The Cartographer 3D SLAM trajectory and the ground truth data were extracted as mentioned in 6.2.9. Therefore one can see that the Cartographer 3D SLAM trajectory is accurate until the wall of the building in the map starts getting curved. After this moment, it starts to drift away with time. One more result that can be inferred from it is that the quality of the ground truth data is also not very accurate, as it has a lot of noisy values and outliers.

From 7.2, one can infer that the Cartographer 3D SLAM trajectory in its x-axis remains pretty accurate for the whole course of the robot. Though, its y-axis accumulates errors over the time in the region where the map is not accurate. But, this is not true when the Cartographer 3D

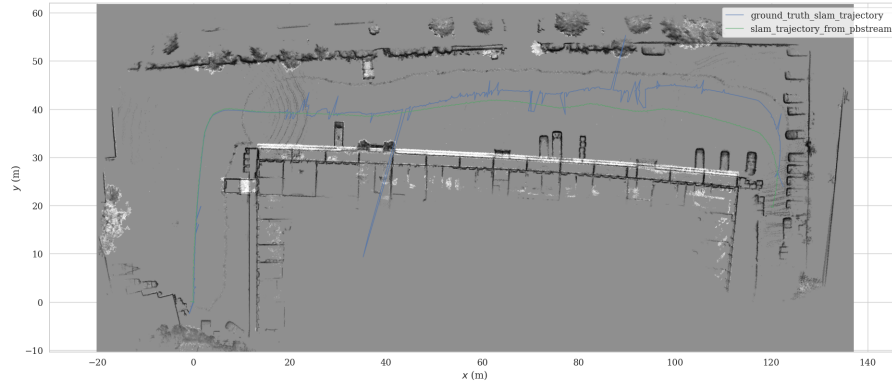


Figure 7.1: Map Comparison of the Cartographer 3D SLAM and the Ground Truth Trajectory

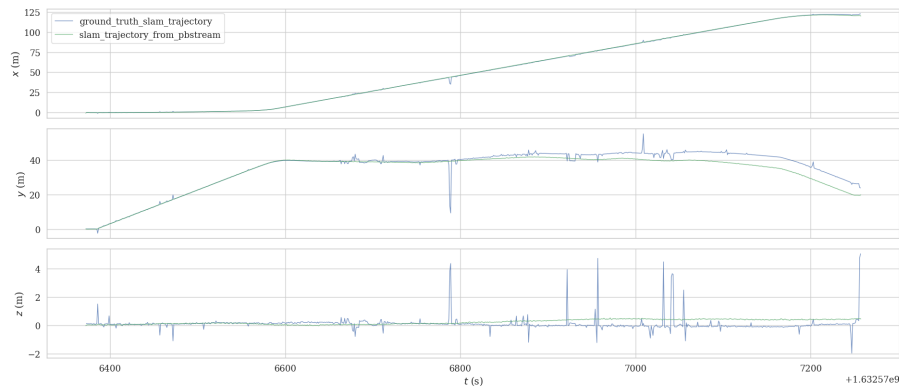


Figure 7.2: Axis-Wise Comparison of the Cartographer 3D SLAM and the Ground Truth Trajectory

SLAM and the ground truth trajectories are rotated by thirty degrees anti-clockwise about the z-axis as shown in 7.3. Here, it is seen that its x-axis also accumulates errors over the time after the quality of the map starts to deteriorate. Therefore, its x-axis is accurate, only due to the chosen driving path of the robot in the map, which is quite parallel to the x-axis of the map in that region. The linear z-axis in the Cartographer 3D SLAM trajectory till the map is accurate shows that the ground of the point cloud map made by the Cartographer 3D SLAM algorithm is quite flat. The z-axis also shows that the ground truth data in its z-axis is not accurate due to the reasons mentioned in 6.1.2. The times in these plots were the ROS times when the Cartographer 3D SLAM and 3D localization algorithms were performed. Also these times were the ROS times when the sensor data was being captured in the bags. This is because the Cartographer 3D SLAM and the 3D localization algorithms perform at the times which they receive from the sensor data in the bags, which is equivalent to performing them online as discussed in 6.2.1.

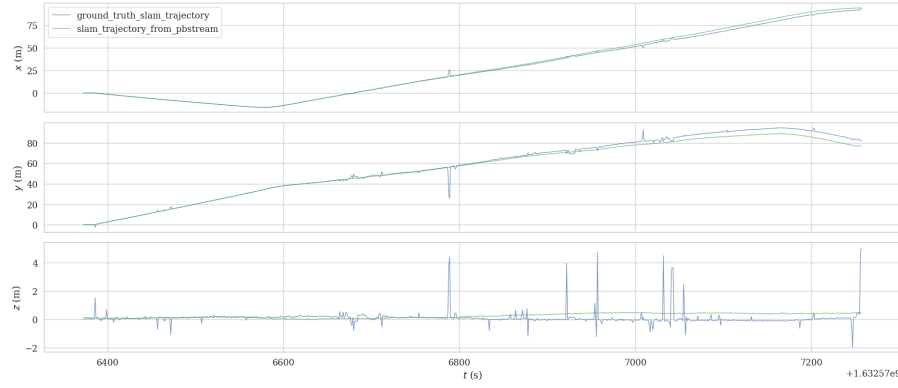


Figure 7.3: Axis-Wise Comparison of the Cartographer 3D SLAM with the Ground Truth Trajectory after their Rotation by Thirty Degrees about the Z-Axis Anti-Clockwise

## 7.1.2 Absolute Pose Errors

3D APE (absolute pose error) is the 3D euclidean distance between the positions of a trajectory from its ground truth which have the nearest matching timestamps. In this application, the nearest matching timestamps were within 0.5 seconds. This was because the frequency of the ground truth data was one hertz and the therefore the Cartographer 3D SLAM and the 3D localization algorithms' trajectories were also compared with it at each one seconds.

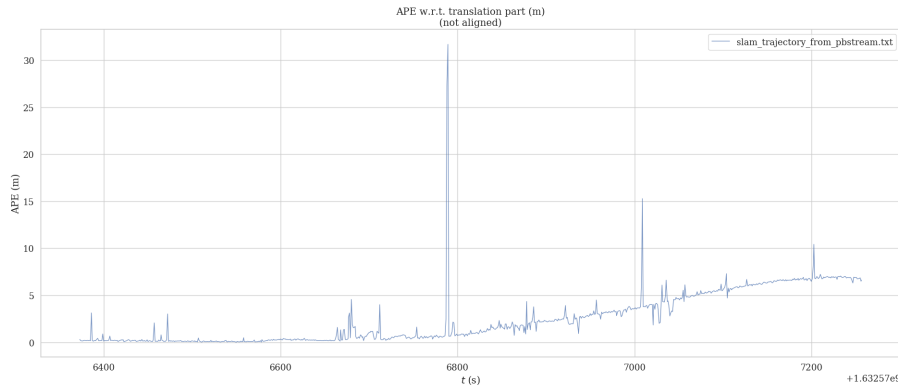


Figure 7.4: Raw Distribution of the 3D APEs in the Cartographer 3D SLAM Trajectory

As seen in 7.4, the 3D APEs in the Cartographer 3D SLAM trajectory were accumulated over the time after the accuracy of the map starts deteriorating which is also discussed in 7.1.1. Before that moment, till about  $t = 1632576800$  seconds the 3D APEs in it were pretty close to zero.

The APEs in the Cartographer 3D SLAM and the 3D localization algorithms were also computed in 2D by neglecting the z-axis component by putting it as zero in the ground truth, Cartographer 3D SLAM and the 3D localization algorithms' trajectories. This was because of the following reasons:

|         | rmse     | mean     | median   | std      | min      | max    |
|---------|----------|----------|----------|----------|----------|--------|
| 3D SLAM | 3.775599 | 2.502136 | 1.326008 | 2.827448 | 0.013703 | 31.685 |

Table 7.1: Statistics of the 3D APEs in the Cartographer 3D SLAM Trajectory in meters

|         | rmse     | mean     | median   | std      | min      | max       |
|---------|----------|----------|----------|----------|----------|-----------|
| 2D SLAM | 3.695398 | 2.425209 | 1.247578 | 2.788248 | 0.010538 | 31.400878 |

Table 7.2: Statistics of the 2D APEs in the Cartographer 2D SLAM Trajectory in meters

1. The z-axis in the ground truth data was not accurate as discussed in 7.1.1.
2. The z-axis of the trajectories of the 3D localization algorithms was either not accurate or had an offset from the z-axis of the ground truth trajectories which is discussed in 7.2.1.
3. The robot used in this application was a tracked wheel robot which was driven on the ground and not in the air.

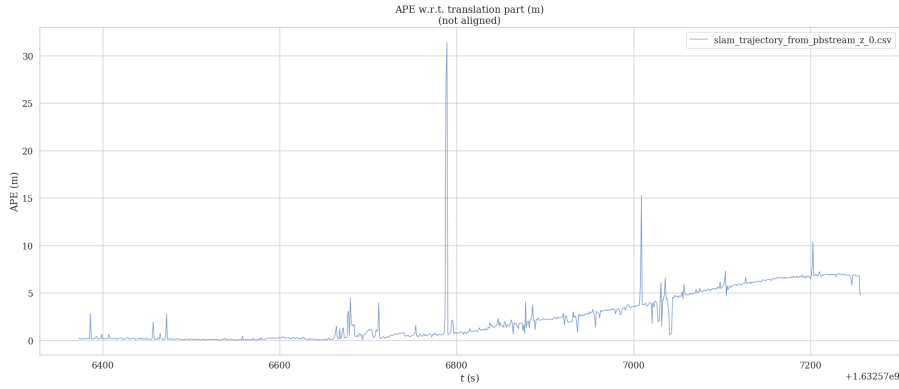


Figure 7.5: Raw Distribution of the 2D APEs in the Cartographer 2D SLAM Trajectory

After projecting the Cartographer 3D SLAM and the ground truth trajectories to 2D as seen in 7.5, it is observed that the distribution of the 2D APEs remain quite the same as its 3D counterpart, shown in 7.4. They also show similar growth in the 2D APEs as its 3D counterpart after  $t = 1632576800$  seconds when the quality of the map starts deteriorating.

The statistics of the APEs in the Cartographer 3D SLAM trajectory and its 2D projection in meters, given in 7.1 and 7.2 respectively, are explained and interpreted as follows:

**rmse** It is the root mean squared APE, computed by taking the square root of the average of the squared APEs. It is about 3.76 meters for the Cartographer 3D SLAM trajectory and about 3.7 meters for its 2D counterpart. Both of these rmse are quite high.

**mean** It is the average of the APEs. It is about 2.5 meters for the Cartographer 3D SLAM trajectory and about 2.43 meters for its 2D counterpart. Both of these mean APEs are quite high.

**median** It is the middle value in a dataset which divides the data sorted in ascending order into lower and upper halves. It is about 1.33 meters for the Cartographer 3D SLAM trajectory and about 1.25 meters for its 2D counterpart. Both of these median APEs are quite high.

**std** It is the deviation of the dataset from its mean. It is calculated as the square root of the variance. In this application the variance is calculated as the taking the difference between each of the APEs of a 3D localization algorithm and its mean, squaring them, adding them and dividing the sum by one less than the number of APEs. If the distribution of the APEs is in the form of a normal distribution, then one can say that about 99.7 % of the APEs lie within the three standard deviations. Therefore if the standard deviation is low, then one can know the range of the distribution of the APEs by adding and subtracting three times the standard deviation from the mean. By knowing this, one will know the range of the errors in the localization and then can subtract this error from the localization provided by a localization algorithm for their application. The standard deviation of the distribution of the APEs is about 2.83 meters for the Cartographer 3D SLAM trajectory and about 2.79 meters for its 2D counterpart. Both of these standard deviations of the distribution of the APEs are quite high.

**min** It is the minimum APE. It is about 0.01 meters for both the Cartographer 3D SLAM trajectory and its 2D counterpart, which shows that they are very low and the Cartographer SLAM trajectory is very accurate in some regions of the map.

**max** It is the maximum APE. It is about 31.69 meters for the Cartographer 3D SLAM trajectory and about 31.4 meters for its 2D counterpart. But both of these maximum APEs are wrongly computed as some of the ground truth positions are inside the building where the robot was not driven and are outliers. They are caused by the inaccurate ground truth data when the robot was driven near to the buildings as discussed in 6.2.1.

The six statistics described above show that the APEs in the Cartographer 3D SLAM trajectory and its 2D counterpart are all quite close to each other. This shows that excluding the z-axis in the computations of the APEs does not improve the quality of the Cartographer 3D SLAM results.

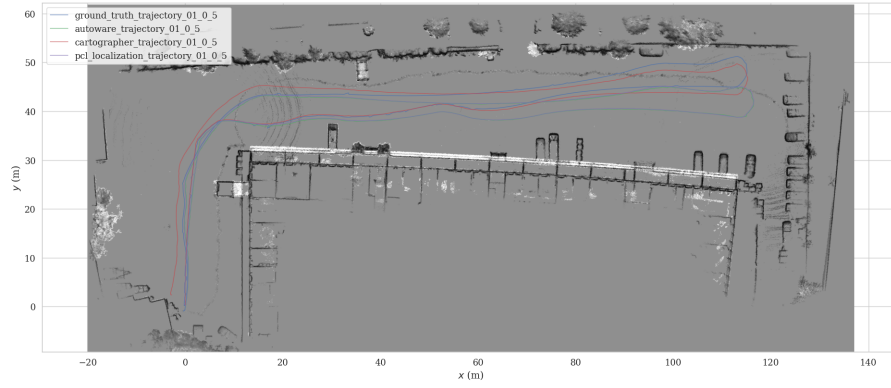


Figure 7.6: Map Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the No Obstacles 3D Localization Challenge

## 7.2 3D Localization Challenges

### 7.2.1 No Obstacles

#### Trajectories Comparisons

The Google Cartographer 3D localization algorithm, Autoware.Auto 3D NDT Localization Algorithm and the `pcl_localization_ros2` 3D PCL NDT Localization Algorithm will be further called as Cartographer, Autoware and the `pcl_localization` respectively in this master thesis. In this 3D localization challenge it is clearly seen in 7.6 that all the trajectories made by the 3D localization algorithms are quite accurate until the map is accurate. After this moment, they start to accumulate errors over the time due to the deteriorating quality of the map far away from its origin as discussed in 7.1. As seen in this plot, the trajectories of the Autoware and the `pcl_localization` are almost overlapping each other for the whole course of the robot, which is due to the same underlying 3D NDT algorithm as discussed in 6.2.11. They only differ a bit when the robot is taking a U-turn at the end of the map. But they accumulate more error far away from the origin of the map than the trajectory of the Cartographer. As visually perceived from this plot, the Cartographer is performing the best localization among these three tested 3D localization algorithms until the map is accurate. Though, when the robot takes a U-turn at the end of map and again returns to the region where the map is accurate, the Autoware and the `pcl_localization` perform better localization than the Cartographer as seen visually in this plot. In this region, the trajectory of the Cartographer drifts away from the trajectory of the ground truth.

It can be seen in 7.7 that the x-axis of the trajectories of all the 3D localization algorithms looks pretty accurate for the whole course of the robot, except that of the Cartographer which drifts away in the end. Though, this conclusion is not true as when all these trajectories are rotated by thirty degrees about the z-axis anti-clockwise, then the x-axis also accumulates errors for all the three tested 3D localization algorithms as seen in 7.8. Therefore, the x-axis of all the three tested 3D localization algorithms is accurate, only due to the chosen driving course of the robot

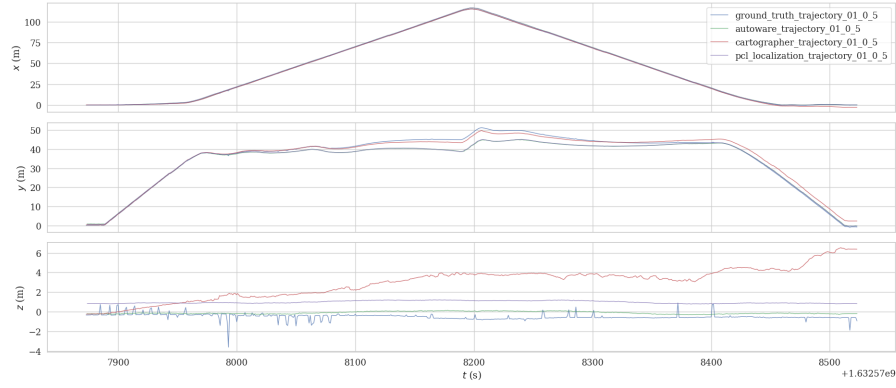


Figure 7.7: Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the No Obstacles 3D Localization Challenge

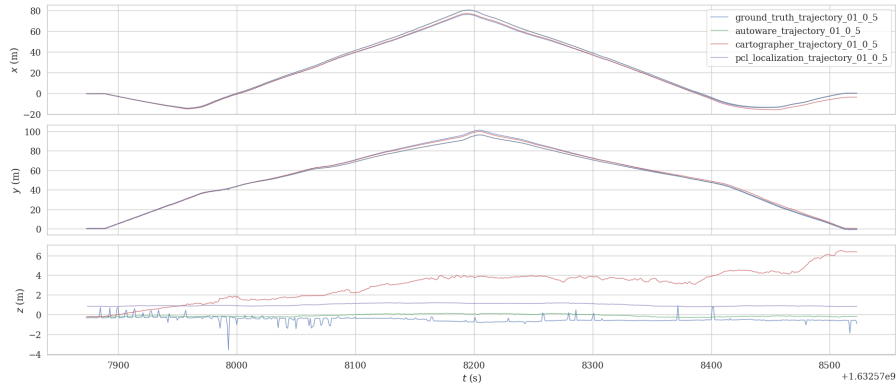


Figure 7.8: Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories after their Rotation by Thirty Degrees about the Z-Axis Anti-Clockwise in the No Obstacles 3D Localization Challenge

in the map, which is quite parallel to the x-axis of the map in that region.

The y-axis in 7.7 accumulates errors in all the three tested 3D localization algorithms in the region where the map is not accurate. The trajectory of the Cartographer accumulates less errors in their y-axis than the trajectory of the Autoware and the pcl\_localization. But when the robot returns to the region of the map where it again becomes accurate, the y-axis of the trajectory of the Autoware and the pcl\_localization overlaps with the y-axis of the trajectory of the ground truth. But from that location till the end of the course of the robot in the map, the y-axis in the trajectory of the Cartographer drifts away from the y-axis of the trajectory of the ground truth. By seeing 7.8, the reader of this master thesis might be confused to see the very accurate y-axis of the trajectory of all the three tested 3D localization algorithms when all these trajectories are rotated by thirty degrees about the z-axis anti-clockwise. But this is not true, due to the same reason discussed in the previous paragraph about the chosen driven course of the robot in the



map.

From 7.7, it is clearly visible that the trajectories in the z-axis are not accurate in the ground truth and the 3D localization algorithms. The z-axis of the trajectories of the ground truth are not accurate due to the reason mentioned in 6.1.2. But, though the z-axis of the trajectory of the Cartographer has a lot of noise and has a very high range, the z-axis of the trajectories of the Autoware and the pcl\_localization are quite parallel to themselves and to the z-axis of the trajectory of the ground truth. The offset in the z-axis of the trajectory of the ground truth is less for the Autoware and more for the pcl\_localization.

## APEs Distributions

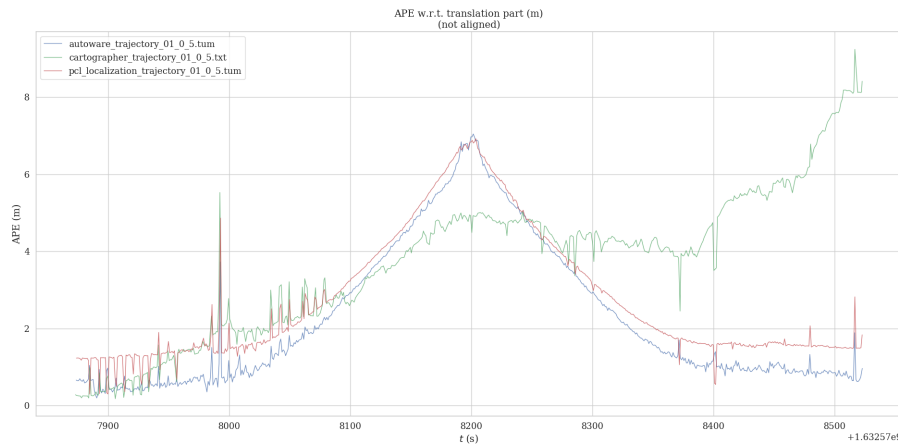


Figure 7.9: Raw Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the No Obstacles 3D Localization Challenge

The raw distributions of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge are given in 7.9. Their interpretation is as follows:

1. The pattern of the 3D APEs is quite the similar for the Autoware and the pcl\_localization and is symmetrical about  $t = 1632578200$  seconds where the robot takes a U-turn at the end of the map.
2. The Autoware and the pcl\_localization start accumulating 3D APEs over the time after about  $t = 1632578000$  seconds where the quality of the map starts deteriorating.
3. But the Cartographer starts accumulating 3D APEs after the quality of the map starts deteriorating and the distribution of its 3D APEs does not decrease when the robot takes a U-turn at the end of the map at about  $t = 1632578200$  seconds. Though, in the region where the map is not accurate, the Cartographer accumulates less 3D APEs over the time than the Autoware and the pcl\_localization.

4. The pcl\_localization has the highest 3D APEs till the map is accurate. In this region of the map, the Autoware performs the best 3D localization among these three tested 3D localization algorithms. Also in this region of the map, the 3D APEs of the Cartographer is between that of the Autoware and the pcl\_localization.
5. When the robot return after making a U-turn at the end of the map to region where the map is again accurate at about  $t = 1632578400$  seconds, the Autoware and the pcl\_localization have almost the similar values of the 3D APEs as they were earlier in that region of the map. But in this region, the trajectory of the Cartographer accumulates higher 3D APEs than that of the Autoware and the pcl\_localization.
6. As visually perceived from this plot, the Autoware mostly performs the best 3D localization among these three tested 3D localization algorithms.

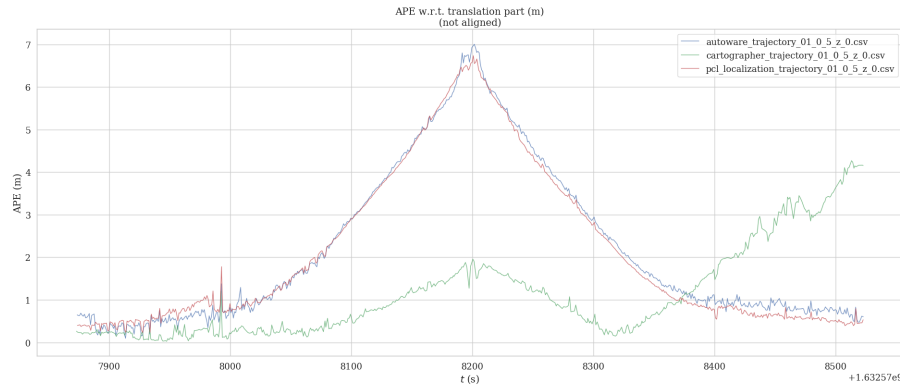


Figure 7.10: Raw Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the No Obstacles 2D Localization Challenge

As the 2D localization is of more importance in this application as discussed in 7.1.2, the raw distributions of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge are shown in 7.10. Their interpretation is as follows:

1. The Autoware and the pcl\_localization have a similar pattern of the distribution of their 2D APEs as it was in their 3D counterpart discussed above. But now they are closer to each other when the distributions of their 2D APEs is compared than in their 3D counterpart.
2. The 2D APEs of the distribution of the Cartographer are quite less than their 3D counterpart as discussed above. It also performs better 2D localization than the Autoware and the pcl\_localization for the whole course of the robot in the map, except when the robot comes back to the region of the map where it is again accurate at about  $t = 1632578400$  seconds. In this region of the map it accumulates higher 2D APEs over the time than the Autoware and the pcl\_localization.

3. Though overall it is hard to choose a winner among these three tested 3D localization algorithms in this 2D localization challenge by just visualizing this plot.

The distributions of the APEs for all the three tested 3D localization algorithms with the ground truth data in all the 3D localization challenges are also represented in the form of boxplots. A boxplot is used to represent the spread of the data through its quartiles. More information about a boxplot can be found in [95] and [96]. A boxplot has the following five statistics:

**Minimum** It is the minimum value in the dataset excluding the outliers. Here, the outliers are the values which are below 1.5 times the interquartile range below the first quartile. The interquartile range is the difference between the third and the first quartile.

**Maximum** It is the maximum value in the dataset excluding the outliers. Here, the outliers are the values which are above 1.5 times the interquartile range above the third quartile.

**Median** It is the median or the middle value of the dataset.

**First quartile** It is median of the lower half of the dataset.

**Third quartile** It is the median of the upper half of the dataset

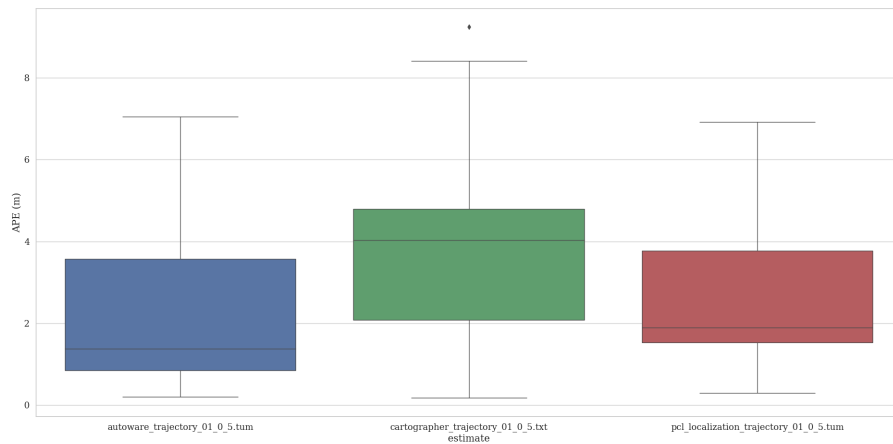


Figure 7.11: Boxplot of the Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the No Obstacles 3D Localization Challenge

The interpretation of the boxplot of the distributions of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge is shown in 7.11. Its interpretation is as follows:

1. The outliers are represented as dots.

2. The Cartographer has the lowest minimum and the pcl\_localization has the lowest maximum 3D APE among these three tested 3D localization algorithms.
3. The range of the distribution of the 3D APEs excluding the outliers is the smallest for the pcl\_localization and is the largest for the Cartographer among these three tested 3D localization algorithms.
4. The distribution of the 3D APEs for the Cartographer is negatively skewed, which depicts that it is concentrated on the right (or above) of the figure.
5. The distributions of the 3D APEs for Autoware and the pcl\_localization are positively skewed, which depicts that they are concentrated on the left (or below) of the figure.
6. Though overall, it is hard to choose a winner among these three tested 3D localization algorithms by just visualizing this boxplot.

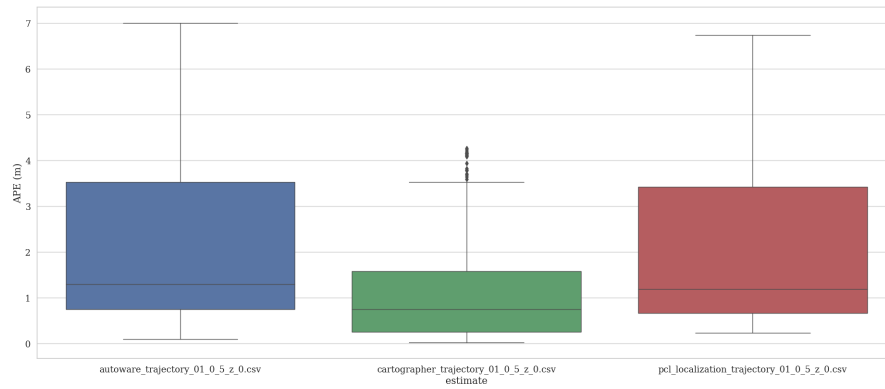


Figure 7.12: Boxplot of the Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the No Obstacles 2D Localization Challenge

As the 2D localization is of more importance in this application as discussed in 7.1.2, the boxplot of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge is given in 7.12. Its interpretation is as follows:

1. The Cartographer has the lowest minimum and the lowest maximum 2D APE among these three tested 3D localization algorithms.
2. The range of the distribution of the 2D APEs excluding the outliers is the smallest for the pcl\_localization and is the largest for the Cartographer among these three tested 3D localization algorithms.
3. The distribution of the 2D APEs for all of these 3D localization algorithms are positively skewed, i.e., they are concentrated on the left (or below) of the figure.

| No Obstacles 3D  | rmse     | mean     | median   | std      | min      | max      |
|------------------|----------|----------|----------|----------|----------|----------|
| Autoware         | 2.865472 | 2.210587 | 1.294598 | 1.823248 | 0.196344 | 7.046964 |
| Cartographer     | 4.391242 | 3.880421 | 4.154647 | 2.055563 | 0.176188 | 9.243508 |
| pcl_localization | 3.150857 | 2.696462 | 1.834704 | 1.630029 | 0.296767 | 6.924504 |

Table 7.3: Statistics of the 3D APEs for all the three tested 3D Localization Algorithms in the No Obstacles 3D Localization Challenge in meters

| No Obstacles 2D  | rmse     | mean     | median   | std      | min      | max      |
|------------------|----------|----------|----------|----------|----------|----------|
| Autoware         | 2.816553 | 2.130121 | 1.206352 | 1.842703 | 0.096653 | 7.00283  |
| Cartographer     | 1.717796 | 1.24012  | 0.831166 | 1.188666 | 0.025487 | 4.268055 |
| pcl_localization | 2.75671  | 2.063446 | 1.070003 | 1.828016 | 0.231673 | 6.734505 |

Table 7.4: Statistics of the 2D APEs for all the three tested 3D Localization Algorithms in the No Obstacles 2D Localization Challenge in meters

4. Visually speaking, the Cartographer is the clear winner of this 2D localization challenge among these three tested 3D localization algorithms as it has the lowest minimum and lowest maximum 2D APE and also the lowest range of the distribution of the 2D APEs excluding the outliers among these three tested 3D localization algorithms.

## APEs Statistics

The statistics of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge in meters, given in 7.3, are interpreted as follows:

**rmse** It is the minimum for the Autoware at about 2.87 meters.

**mean** It is the minimum for the Autoware at about 2.21 meters.

**median** It is the minimum for the Autoware at about 1.29 meters.

**std** It is the minimum for the pcl\_localization at about 1.63 meters.

**min** It is the minimum for the Cartographer at about 0.18 meters.

**max** It is the minimum for the pcl\_localization at about 6.92 meters.

As the 2D localization is of more importance in this application as discussed in 7.1.2, the statistics of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge

in meters are given in 7.4. Their interpretation is as follows:

**rmse** It is the minimum for the Cartographer at about 1.72 meters.

**mean** It is the minimum for the Cartographer at about 1.24 meters.

**median** It is the minimum for the Cartographer at about 0.83 meters.

**std** It is the minimum for the Cartographer at about 1.19 meters.

**min** It is the minimum for the Cartographer at about 0.03 meters.

**max** It is the minimum for the Cartographer at about 4.27 meters.

## Winner

The winner of each 3D localization challenge is decided on the basis of the most number of wins in the statistics of the 3D APEs. Therefore, the Autoware wins in this 3D localization challenge as it is the winner of three out of the six statistics of the 3D APEs as discussed in 7.2.1. But as the 2D localization is of importance in this application as discussed in 7.1.2, then the Cartographer wins this 2D localization challenge as it is the winner of all of the six statistics of the 2D APEs as discussed in 7.2.1.

## 7.2.2 High Velocity

### Trajectories Comparisons

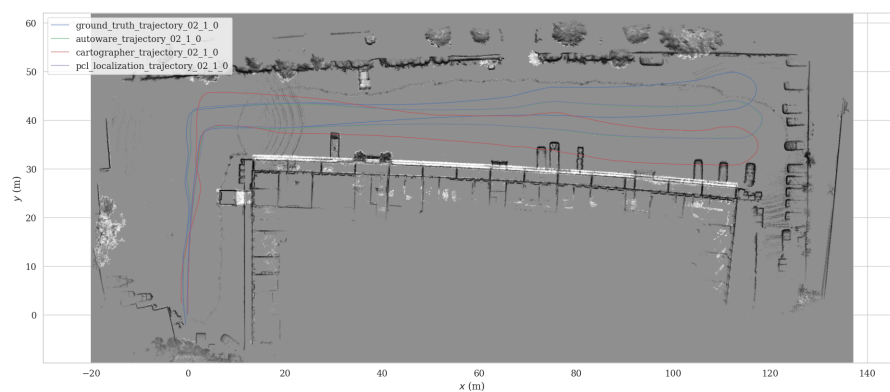


Figure 7.13: Map Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the High Velocity 3D Localization Challenge

In this 3D localization challenge, it is seen in 7.13, that all the trajectories made by the 3D localization algorithms are quite accurate until the map is accurate. After this moment, they start to accumulate errors over the time due to the deteriorating quality of the map far away from its origin as discussed in 7.1. As seen in this plot, the trajectories of the Autoware and the pcl\_localization are almost overlapping each other for the whole course of the robot, except when the robot takes a U-turn at the end of the map, which was also the case for the no obstacles 3D localization challenge, as discussed in 7.2.1. In this challenge, as visually perceived from this plot, the trajectory of the Autoware and the pcl\_localization accumulate less errors over the time in the region where the map is not accurate, than the trajectory of the Cartographer, which is opposite to the no obstacles 3D localization challenge as discussed in 7.2.1. When the robot takes a U-turn at the end of map and returns to the region where the quality of the map is again accurate, the Autoware and the pcl\_localization perform better localization than the Cartographer as visually seen in this plot. In this region of the map, the trajectory of the Cartographer drifts away from the trajectory of the ground truth.

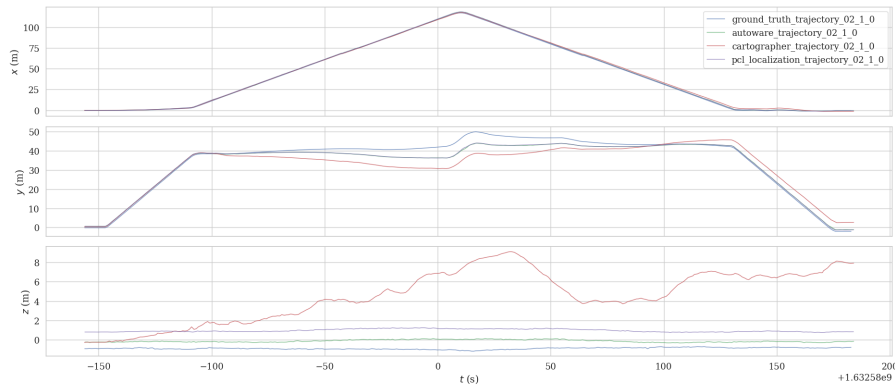


Figure 7.14: Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the High Velocity 3D Localization Challenge

As seen in 7.14, that the x-axis of the trajectories of all the 3D localization algorithms look pretty accurate for the whole course of the robot, even in the region where the map is not accurate, except of the Cartographer which drifts away in the end. Though, this conclusion is not true as when all these trajectories are rotated by thirty degrees about the z-axis anti-clockwise, then the x-axis also accumulates errors for all the 3D localization algorithms as seen in 7.15. Therefore, the x-axis of all the 3D localization algorithms is accurate, only due to the chosen driving course of the robot in the map, which is quite parallel to the x-axis of the map in that region.

The y-axis in 7.14 accumulates errors in all the 3D localization algorithms in the region where the map is not accurate. The trajectory of the Autoware and the pcl\_localization accumulate less errors in their y-axis than the y-axis of the trajectory of the Cartographer. But when the robot returns to the region of the map where it again becomes accurate, the y-axis of the trajectories of the Autoware and the pcl\_localization overlaps with the y-axis of the trajectory of the ground truth. But from that location till the the end of the course of the robot, the y-axis in the trajectory

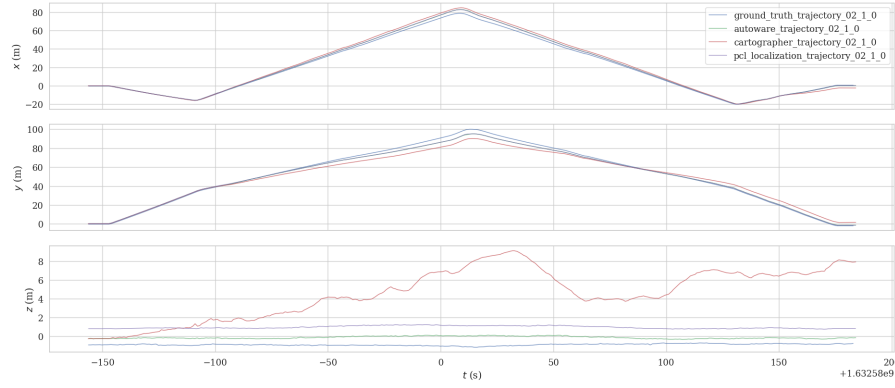


Figure 7.15: Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories after their Rotation by Thirty Degrees about the Z-Axis Anti-Clockwise in the High Velocity 3D Localization Challenge

of the Cartographer drifts away from the y-axis of the trajectory of the ground truth. By seeing 7.8, the reader of this master thesis might be confused to see the very accurate y-axis of the trajectory of the Cartographer when all these trajectories are rotated by thirty degrees about the z-axis anti-clockwise. But this is not true, due to the same reason discussed in the previous paragraph about the chosen driving course of the robot in the map.

From 7.14, it is clearly visible that the trajectories in the z-axis are not accurate of the ground truth and the 3D localization algorithms. But, though the z-axis in the trajectory of the Cartographer has a lot of noise and has a very high range, the z-axis of the trajectories of the Autoware and the pcl\_localization are quite parallel to themselves and to the z-axis of the trajectory of the ground truth, as also seen in 7.2.1. The offset in the z-axis of the trajectory of the ground truth is less for the Autoware and more for the pcl\_localization.

## APEs Distributions

The raw distributions of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge are given in 7.16. Their interpretation is as follows:

1. The distribution of the 3D APEs for the Autoware and the pcl\_localization are quite similar to each other and are symmetrical about the point where the robot takes a U-turn at the end of the map between  $t = 1632580000$  seconds and  $t = 1632580050$  seconds.
2. The Autoware and the pcl\_localization start accumulating 3D APEs over the time after about  $t = 1632579900$  seconds where the quality of the map starts deteriorating.
3. The distribution of the 3D APEs for the Cartographer starts accumulating 3D APEs from the starting pose of the robot only. Though when the robot starts returning to the origin of



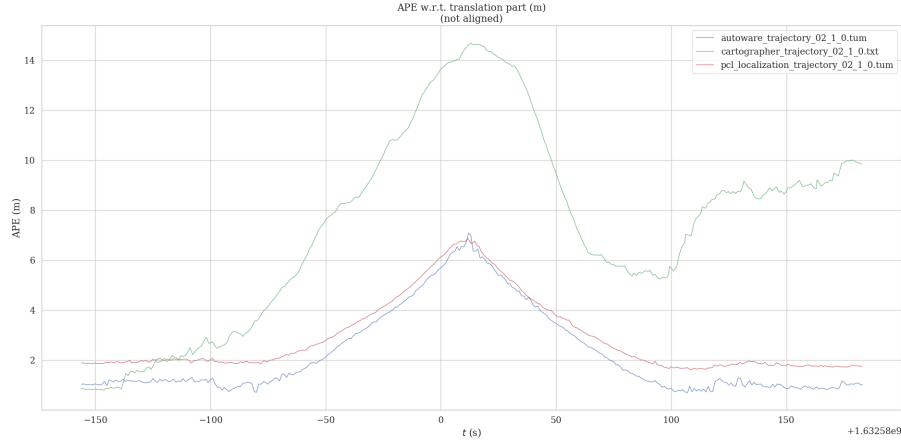


Figure 7.16: Raw Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the High Velocity 3D Localization Challenge

the map after taking a U-turn at the end of the map between  $t = 1632580000$  seconds and  $t = 1632580050$  seconds, the 3D APEs start decreasing. But they again start accumulating the 3D APEs when the map becomes accurate again at about  $t = 1632580100$  seconds.

4. Though, in the region where the map is not accurate, the Cartographer accumulates less 3D APEs over the time than the Autoware and the pcl\_localization.
5. The pcl\_localization has the highest 3D APEs till the map is accurate. In this region of the map, the Autoware performs the best 3D localization among these three tested 3D localization algorithms. Also in this region of the map, the 3D APEs of the Cartographer is between that of the Autoware and the pcl\_localization.
6. When the robot returns after making a U-turn at the end of the map to region where the map is again accurate at about  $t = 1632580100$  seconds, the Autoware and the pcl\_localization have almost the similar values of the 3D APEs as they were earlier in that region of the map. But in this region of the map, the trajectory of the Cartographer accumulates higher 3D APEs than that of the Autoware and the pcl\_localization.
7. As visually perceived from this plot, the Autoware mostly performs the best 3D localization among these three tested 3D localization algorithms.

As the 2D localization is of more importance in this application as discussed in 7.1.2, the raw distributions of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge are given in 7.17. Their interpretation is as follows:

1. The distribution of the 2D APEs of the Autoware and the pcl\_localization are quite similar

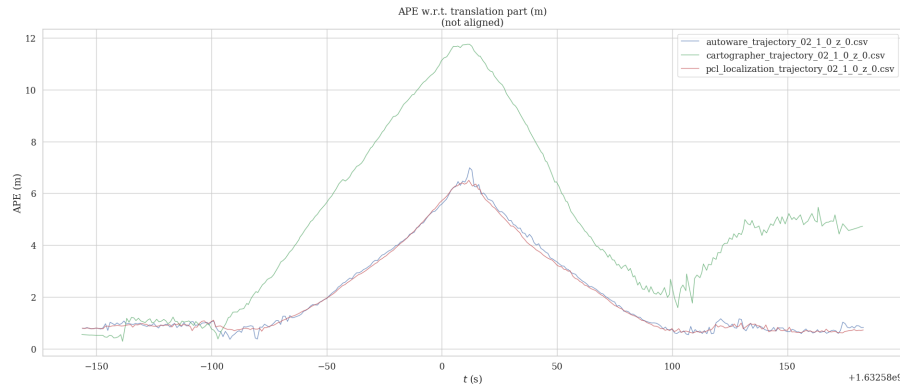


Figure 7.17: Raw Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the High Velocity 2D Localization Challenge

and close to each other, as it was in their 3D counterpart discussed above. They are symmetrical about the point where the robot takes a U-turn at the end of the map between  $t = 1632580000$  seconds and  $t = 1632580050$  seconds.

2. The distribution of the 2D APEs for the Cartographer are quite less than their 3D counterpart as discussed above. It has quite comparable 2D APEs compared with that of the Autoware and the pcl\_localization where the quality of the map is accurate till about  $t = 1632579900$  seconds, which was not the case for its 3D counterpart. After that point, it starts accumulating 2D APEs till the robot starts taking a U-turn at the end of the map. Though when the robot starts returning to the origin of the map after taking a U-turn at the end of the map, its 2D APEs start decreasing. But it again starts accumulating the 2D APEs when the map becomes accurate again at about  $t = 1632580100$  seconds. In this region of the map it accumulates higher 2D APEs over the time than that of the Autoware and the pcl\_localization.
3. Though overall, it is hard to choose a winner among these three tested 3D localization algorithms by just visualizing this plot.

The interpretation of the boxplot of the distributions of the 3D APEs in the High Velocity 3D localization challenge shown in 7.18 is as follows:

1. The Autoware has the lowest minimum 3D APE and the pcl\_localization has the lowest maximum 3D APE among these three tested 3D localization algorithms.
2. The range of the distribution of the 3D APEs excluding the outliers is the smallest for the pcl\_localization and is the largest for the Cartographer among these three tested 3D localization algorithms.
3. The distribution of the 3D APEs for the Cartographer is negatively skewed, which depicts

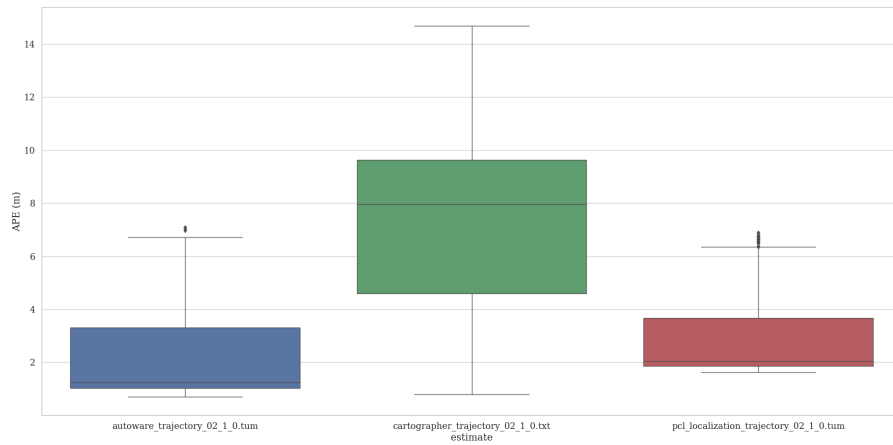


Figure 7.18: Boxplot of the Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the High Velocity 3D Localization Challenge

that it is concentrated on the right (or above) of the figure.

4. The distributions of the APEs for Autoware and the pcl\_localization are positively skewed, which depicts that they are concentrated on the left (or below) of the figure.
5. Though overall, it is hard to choose a winner among these three tested 3D localization algorithms by just visualizing this boxplot.

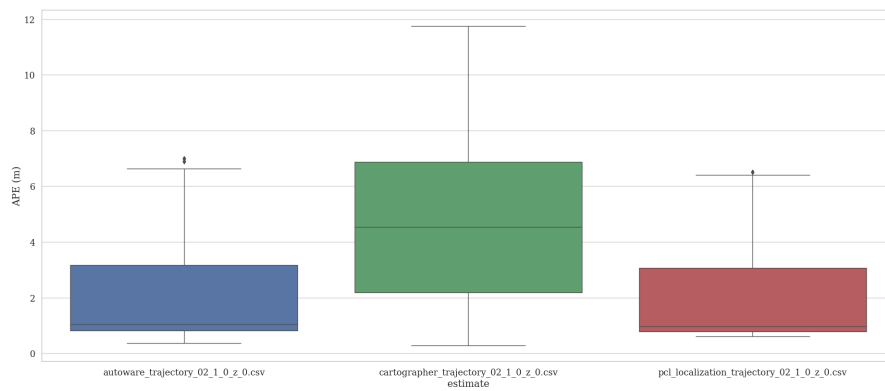


Figure 7.19: Boxplot of the Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the High Velocity 2D Localization Challenge

As the 2D localization is of more importance in this application as discussed in 7.1.2, the boxplot of the distributions of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge is shown in 7.19. Its interpretation is as follows:

1. The Cartographer has the lowest minimum and the pcl\_localization has the lowest maximum 2D APE among these three tested 3D localization algorithms.

| High Velocity 3D | rmse     | mean     | median   | std      | min      | max       |
|------------------|----------|----------|----------|----------|----------|-----------|
| Autoware         | 2.767635 | 2.18997  | 1.199956 | 1.692286 | 0.700802 | 7.086727  |
| Cartographer     | 8.487677 | 7.559695 | 8.342798 | 3.858974 | 0.799499 | 14.681592 |
| pcl_localization | 3.199374 | 2.841225 | 2.010399 | 1.470862 | 1.628157 | 7.258601  |

Table 7.5: Statistics of the 3D APEs for all the three tested 3D Localization Algorithms in the High Velocity 3D Localization Challenge in meters

2. The range of the distribution of the 2D APEs is the largest for the Cartographer and the smallest for the pcl\_localization among these three tested 3D localization algorithms.
3. The distributions of the 2D APEs for Autoware and the pcl\_localization are positively skewed, which depicts that they are concentrated on the left (or below) of the figure.
4. Though overall, it is hard to choose a winner among these three tested 3D localization algorithms by just visualizing this boxplot.

### APEs Statistics

The statistics of the 3D APEs for this 3D localization challenge in meters, given in 7.5, are interpreted as follows:

**rmse** It is the minimum for the Autoware at about 2.77 meters.

**mean** It is the minimum for the Autoware at about 2.19 meters.

**median** It is the minimum for the Autoware at about 1.2 meters.

**std** It is the minimum for the pcl\_localization at about 1.47 meters.

**min** It is the minimum for the Autoware at about 0.7 meters.

**max** It is the minimum for the Autoware at about 7.09 meters.

As the 2D localization is of more importance in this application as discussed in 7.1.2, the statistics of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge in meters are given in 7.6. Their interpretation is as follows:

**rmse** It is the minimum for the pcl\_localization at about 2.61 meters.

**mean** It is the minimum for the pcl\_localization at about 1.97 meters.

| High Velocity 2D | rmse     | mean     | median   | std      | min      | max       |
|------------------|----------|----------|----------|----------|----------|-----------|
| Autoware         | 2.6553   | 2.013157 | 0.99775  | 1.731421 | 0.364382 | 6.986306  |
| Cartographer     | 5.782187 | 4.821745 | 4.672539 | 3.19131  | 0.157437 | 11.755412 |
| pcl_localization | 2.610251 | 1.973087 | 0.942825 | 1.708899 | 0.604418 | 6.907481  |

Table 7.6: Statistics of the 2D APEs for all the three tested 3D Localization Algorithms in the High Velocity 2D Localization Challenge in meters

**median** It is the minimum for the pcl\_localization at about 0.94 meters.

**std** It is the minimum for the pcl\_localization at about 1.71 meters.

**min** It is the minimum for the Cartographer at about 0.16 meters.

**max** It is the minimum for the pcl\_localization at about 6.91 meters.

## Winner

The Autoware is the winner of this 3D localization challenge as it is the winner of five out of the six statistics of the 3D APEs. But as discussed in 7.1.2, the 2D localization is of more importance, therefore the pcl\_localization is the winner of this 2D localization challenge as it wins five out the six statistics of the 2D APEs as discussed in 7.2.2.

## 7.2.3 Fog

### Trajectories Comparisons

From 7.20, it is clearly seen that the fog was not able to deteriorate the accuracy of the localization for any of the tested 3D localization algorithms. Though, the difference in the trajectories of these 3D localization algorithms with themselves is more visible due to the smaller resolution of the map (2.5 meters in the x-axis and 1 meters in the y-axis) in this plot than in the no obstacles and the high velocity 3D localization challenge (20 meters in x-axis and 10 meters in the y-axis), as seen in 7.6 and 7.13.

As seen in 7.21, all the trajectories are a bit noisy in their y-axis though they are smooth in their x-axis. But this inference is not true when all these trajectories are rotated by thirty degrees about the z-axis anti-clockwise in 7.22. Here, the y-axis of all the trajectories are smooth. Also the reasons they are looking noisy is due to the resolution of this plot (5 meters in the x-axis, 2 meters in the y-axis and 0.5 meters in the z-axis) compared to the no obstacles and the high

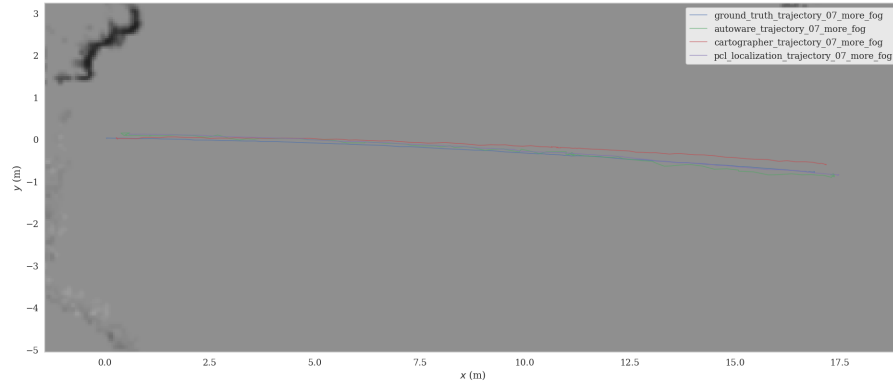


Figure 7.20: Map Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Fog 3D Localization Challenge

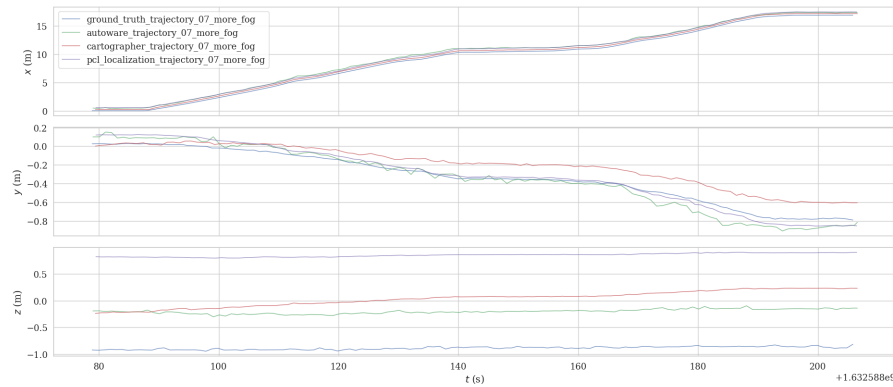


Figure 7.21: Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Fog 3D Localization Challenge

velocity 3D localization challenge (25 meters in the x-axis, 10 meters in the y-axis and 2 meters in the z-axis), as seen in 7.7 and 7.14.

## APEs Distributions

The raw distributions of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge are shown in 7.23. Their interpretation is as follows:

1. The pcl\_localization has the highest 3D APEs among these three tested 3D localization algorithms but they remain quite constant.
2. The Cartographer has the lowest 3D APEs among these three tested 3D localization algorithms till about  $t = 1632588140$  seconds. But after that point the Autoware has the lowest 3D APEs.

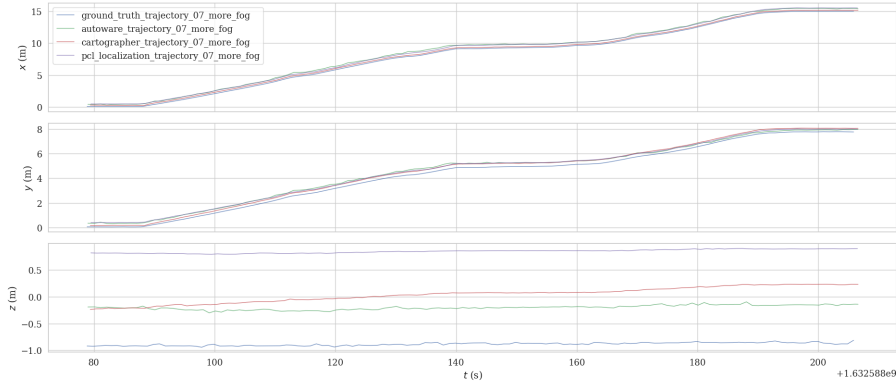


Figure 7.22: Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories after their Rotation by Thirty Degrees about the Z-Axis Anti-Clockwise in the Fog 3D Localization Challenge

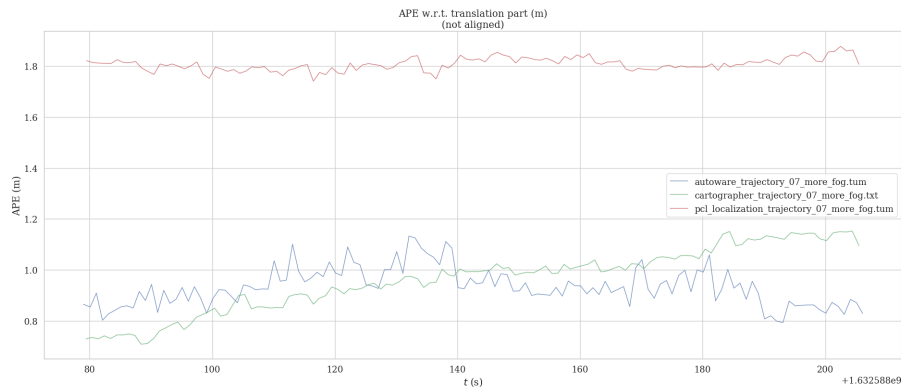


Figure 7.23: Raw Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Fog 3D Localization Challenge

3. Though overall it is hard to choose a winner among these three tested 3D localization algorithms by just visualizing this plot.

As the 2D localization is of more importance in this application as discussed in 7.1.2, the raw distributions of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge are shown in 7.24. Their interpretation is as follows:

1. The Cartographer has the lowest 2D APEs among these three tested 3D localization algorithms in general.
2. The Autoware has the highest 2D APEs among these three tested 3D localization algorithms in general.
3. The pcl\_localization has its 2D APEs between that of the Cartographer and the Autoware

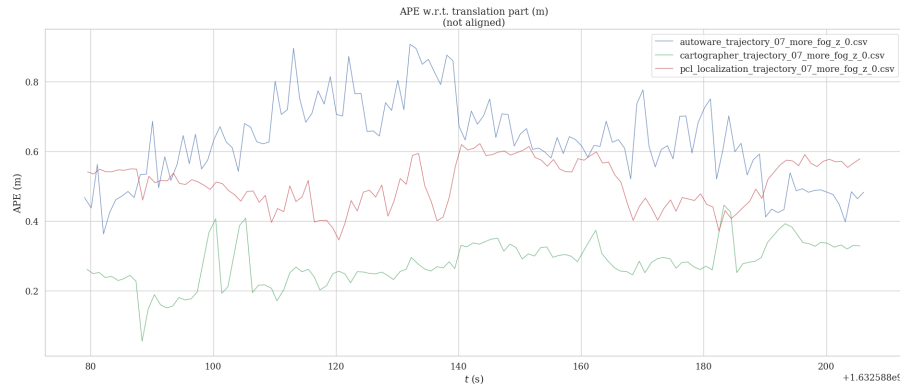


Figure 7.24: Raw Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the Fog 2D Localization Challenge

in general.

4. As visually perceived from this plot, the Cartographer performs the best 2D localization among these three tested 3D localization algorithms in this 2D localization challenge.

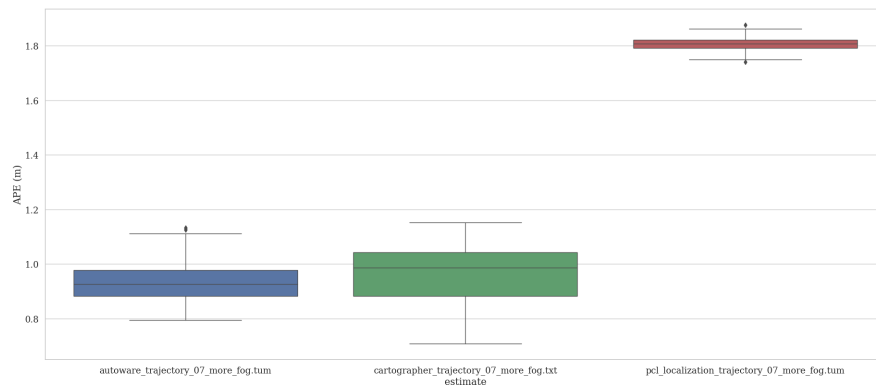


Figure 7.25: Boxplot of the Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Fog 3D Localization Challenge

The interpretation of the boxplot of the distributions of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge shown in 7.25 is as follows:

1. The Cartographer has the lowest minimum and the Autoware has the lowest maximum 3D APE among these three tested 3D localization algorithms.
2. The range of the distribution of the 3D APEs excluding the outliers is the smallest for the pcl\_localization and the largest for the Cartographer among these three tested 3D localization algorithms.



3. The distribution of the 3D APEs for the Cartographer are negatively skewed, which depicts that its distribution of the 3D APEs is concentrated on the right (or above) of the figure.
4. Though overall, it is hard to choose a winner among these three 3D localization algorithms by just visualizing this boxplot.

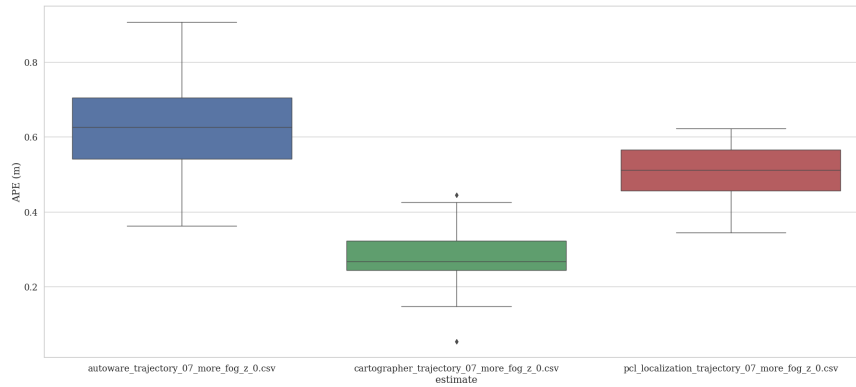


Figure 7.26: Boxplot of the Distributions of the 2D APEs for All the Three Tested 3D Localization Algorithms in the Fog 2D Localization Challenge

As the 2D localization is of more importance in this application as discussed in 7.1.2, the interpretation of the boxplot of the distributions of the 2D APEs in the Fog 2D localization challenge shown in 7.26 is as follows:

1. The Cartographer has the lowest minimum and the lowest maximum 2D APE among these three tested 3D localization algorithms.
2. The range of the distribution of the 2D APEs excluding the outliers is the smallest for both the Cartographer and the pcl\_localization and the largest for the Autoware among these three tested 3D localization algorithms.
3. The distribution of the 2D APEs for the Cartographer is positively skewed, which depicts that its distribution of the 2D APEs is concentrated on the left (or below) of the figure.
4. As visually perceived from this boxplot, the Cartographer performs the best 2D localization among these three tested 3D localization algorithms in this 2D localization challenge. This is because it has the smallest range of the distribution of the 2D APEs excluding the outliers and also the lowest minimum and the lowest maximum 2D APE among these three tested 3D localization algorithms.

| Fog 3D           | rmse     | mean     | median   | std      | min      | max      |
|------------------|----------|----------|----------|----------|----------|----------|
| Autoware         | 0.928579 | 0.925524 | 0.92287  | 0.075268 | 0.793602 | 1.132657 |
| Cartographer     | 0.978745 | 0.9707   | 0.992449 | 0.125234 | 0.708665 | 1.167233 |
| pcl_localization | 1.807596 | 1.807436 | 1.806747 | 0.024079 | 1.740311 | 1.876919 |

Table 7.7: Statistics of the 3D APEs for all the three tested 3D Localization Algorithms in the Fog 3D Localization Challenge in meters

| Fog 2D           | rmse     | mean     | median   | std      | min      | max      |
|------------------|----------|----------|----------|----------|----------|----------|
| Autoware         | 0.628064 | 0.616227 | 0.615154 | 0.121359 | 0.362265 | 0.907179 |
| Cartographer     | 0.285689 | 0.278992 | 0.279839 | 0.061496 | 0.053982 | 0.445275 |
| pcl_localization | 0.517006 | 0.512839 | 0.51825  | 0.065514 | 0.344951 | 0.621804 |

Table 7.8: Statistics of the 2D APEs for all the three tested 3D Localization Algorithms in the Fog 2D Localization Challenge in meters

### APEs Statistics

The statistics of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge in meters, given in 7.7, are interpreted as follows:

**rmse** It is the minimum for the Autoware at about 0.93 meters.

**mean** It is the minimum for the Autoware at about 0.93 meters.

**median** It is the minimum for the Autoware at about 0.92 meters.

**std** It is the minimum for the pcl\_localization at about 0.02 meters.

**min** It is the minimum for the Cartographer at about 0.71 meters.

**max** It is the minimum for the Autoware at about 1.13 meters.

As the 2D localization is of more importance in this application as discussed in 7.1.2, the statistics of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge in meters, given in 7.8, are interpreted as follows:

**rmse** It is the minimum for the Cartographer at about 0.29 meters.

**mean** It is the minimum for the Cartographer at about 0.28 meters.

**median** It is the minimum for the Cartographer at about 0.28 meters.

**std** It is the minimum for the Cartographer at about 0.06 meters.

**min** It is the minimum for the Cartographer at about 0.05 meters.

**max** It is the minimum for the Cartographer at about 0.45 meters.

## Winner

The Autoware wins in this 3D localization challenge as it is the winner of four out of the six statistics of the 3D APEs in it as discussed in 7.2.3. But as discussed in 7.1.2, the 2D localization is of more importance, therefore the Cartographer is the winner of this 2D localization challenge as it wins all of the six statistics of the 2D APEs as discussed in 7.2.2.

## 7.2.4 Pitch

### Trajectories Comparisons

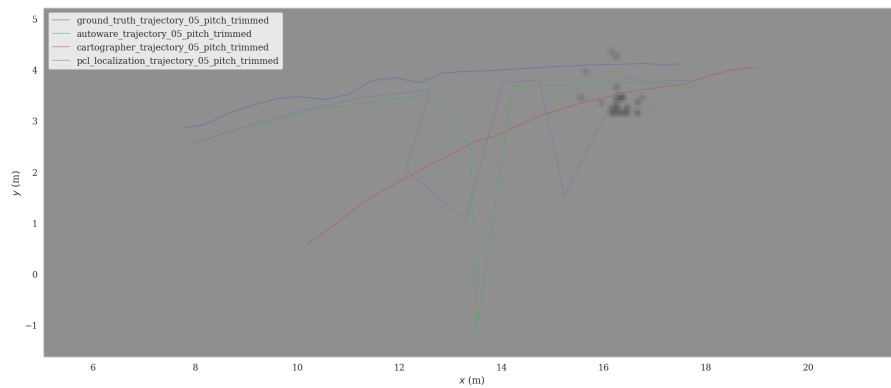


Figure 7.27: Map Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Pitch 3D Localization Challenge

From 7.27, it is clearly seen that the Autoware and the pcl\_localization were able to quickly recover the pose of the robot after it goes over the slope. But the Cartographer was not able to perform localization in this challenge and its trajectory drifted away from the ground truth.

From 7.28, it is seen that the x-axis and the y-axis of the trajectories of the Autoware and the pcl\_localization were disturbed by the slope but they then recovered quickly and were aligned with the x-axis and the y-axis of the trajectory of the ground truth. The x-axis and the y-axis of the trajectory of the Cartographer were not able align with the x-axis and the y-axis of the trajectory of the ground truth. The z-axis of the trajectory of the Autoware and the pcl\_localization are quite parallel to the z-axis of the trajectory of the ground truth and to themselves which is also

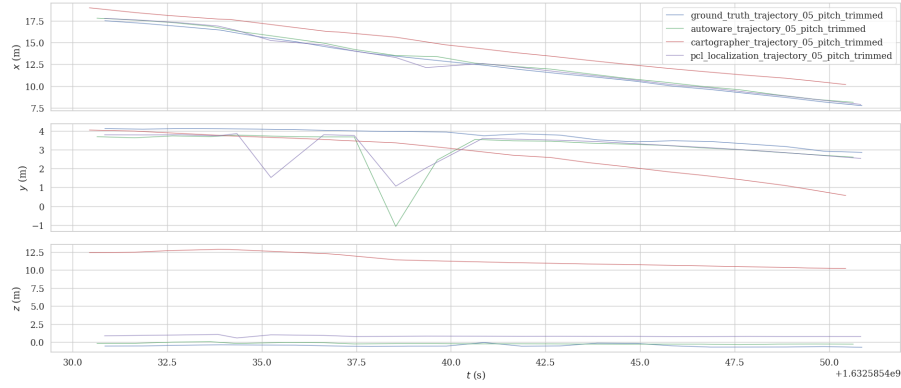


Figure 7.28: Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Pitch 3D Localization Challenge

discussed in 7.2.1. The z-axis in these trajectories are quite parallel to each other, except that of the Cartographer which has a very high range and is inaccurate.

## APEs Distributions

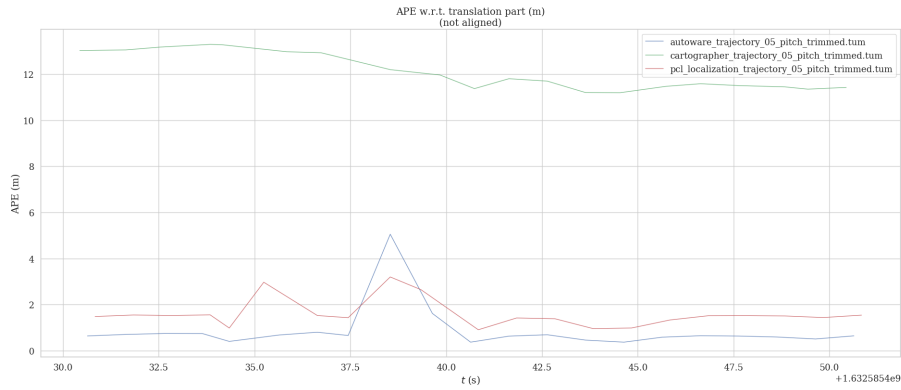


Figure 7.29: Raw Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Pitch 3D Localization Challenge

The raw distributions of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge are shown in 7.29. Its interpretation is as follows:

1. The Cartographer as the highest distribution of the 3D APEs among these three tested 3D localization algorithms.
2. The Autoware has the lowest distribution of the 3D APEs among these three tested 3D localization algorithms in general.

3. The distribution of the 3D APEs of the pcl\_localization is between that of the Autoware and the Cartographer in general.
4. As visually perceived from this plot, the Autoware has the lowest distribution of the 3D APEs among these three tested 3D localization algorithms in general, and therefore is the winner of this 3D localization challenge.

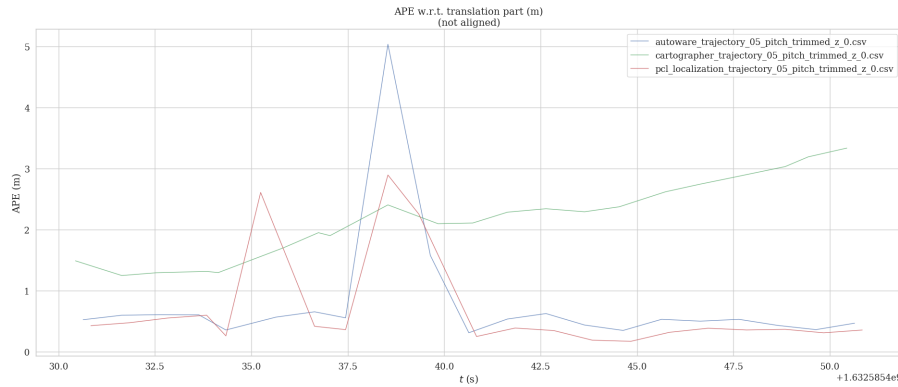


Figure 7.30: Raw Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the Pitch 2D Localization Challenge

As the 2D localization is of more importance in this application as discussed in 7.1.2, the raw distributions of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge are shown in 7.30. Their interpretation is as follows:

1. The Cartographer as the highest distribution of the 2D APEs among these three tested 3D localization algorithms in general.
2. The pcl\_localization has the lowest distribution of the 2D APEs among these three tested 3D localization algorithms in general.
3. The distribution of the 2D APEs of the Autoware is between that of the pcl\_localization and the Cartographer in general.
4. From this plot, one can say that the pcl\_localization is the winner of this 2D localization challenge as it has the lowest distribution of the 2D APEs among these three tested 3D localization algorithms in general.

The interpretation of the boxplot of the distributions of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge shown in 7.31 is as follows:

1. The Autoware has the lowest minimum and the lowest maximum 3D APE among these three tested 3D localization algorithms.

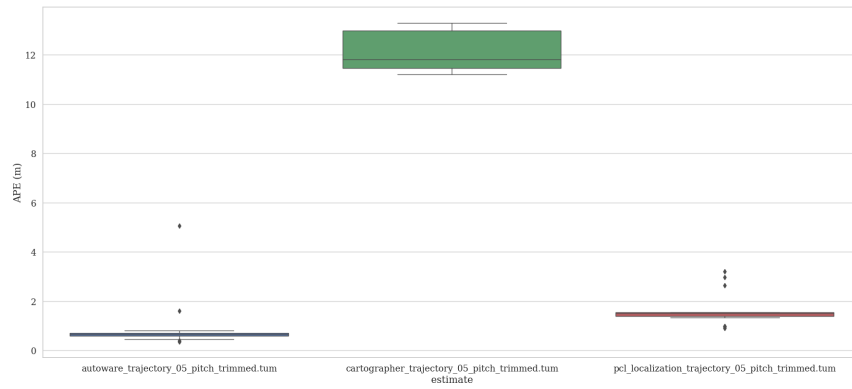


Figure 7.31: Boxplot of the Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Pitch 3D Localization Challenge

2. The range of the distribution of the 3D APEs excluding the outliers is the smallest for the pcl\_localization and the largest for the Cartographer among these three tested 3D localization algorithms.
3. The distribution of the 3D APEs for the Cartographer is positively skewed, which depicts that its distribution of the 3D APEs is concentrated on the left (or below) of the figure.
4. As visually perceived from this boxplot, the Autoware performs the best 3D localization among these three tested 3D localization algorithms as it has the lowest minimum and the maximum 3D APE among them in this 3D localization challenge.

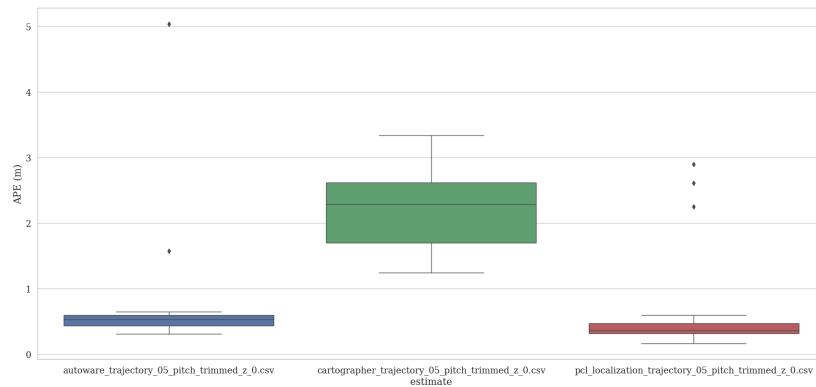


Figure 7.32: Boxplot of the Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the Pitch 2D Localization Challenge

As the 2D localization is of more importance in this application as discussed in 7.1.2, the interpretation of the boxplot of the distributions of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge shown in 7.31 is as follows:

1. The pcl\_localization has the lowest minimum and the lowest maximum 2D APE among

| Pitch 3D         | rmse      | mean      | median    | std      | min       | max       |
|------------------|-----------|-----------|-----------|----------|-----------|-----------|
| Autoware         | 1.296149  | 0.861329  | 0.639493  | 0.968563 | 0.364235  | 5.051593  |
| Cartographer     | 12.158439 | 12.133794 | 11.805251 | 0.773751 | 11.196627 | 13.296335 |
| pcl_localization | 1.693971  | 1.585681  | 1.503318  | 0.595947 | 0.903246  | 3.196612  |

Table 7.9: Statistics of the 3D APEs for all the three tested 3D Localization Algorithms in the Pitch 3D Localization Challenge in meters

these three tested 3D localization algorithms.

2. The range of the distribution of the 2D APEs excluding the outliers is the smallest for the Autoware and the largest for the Cartographer among these three tested 3D localization algorithms.
3. The distributions of the 2D APEs for the Autoware and the Cartographer are negatively skewed, which depicts that its distribution of the 2D APEs is concentrated on the right (or above) of the figure.
4. The distribution of the 2D APEs for the pcl\_localization is positively skewed, which depicts that its distribution of the 2D APEs is concentrated on the left (or below) of the figure.
5. As visually perceived from this boxplot, the pcl\_localization performs the best 2D localization among these three tested 3D localization algorithms in this 2D localization challenge as it has the lowest minimum and the lowest maximum 2D APE among them.

## APEs Statistics

The statistics of the APEs for all the three tested 3D localization algorithms in this 3D localization challenge in meters, given in 7.9, are interpreted as follows:

**rmse** It is the minimum for the Autoware at about 1.3 meters.

**mean** It is the minimum for the Autoware at about 0.86 meters.

**median** It is the minimum for the Autoware at about 0.64 meters.

**std** It is the minimum for the pcl\_localization at about 0.6 meters.

**min** It is the minimum for the Autoware at about 0.36 meters.

**max** It is the minimum for the pcl\_localization at about 3.2 meters.

| Pitch 2D         | rmse     | mean     | median   | std      | min      | max      |
|------------------|----------|----------|----------|----------|----------|----------|
| Autoware         | 1.250059 | 0.767071 | 0.52862  | 0.98704  | 0.307578 | 5.039706 |
| Cartographer     | 2.273617 | 2.185264 | 2.285162 | 0.627659 | 1.24627  | 3.336689 |
| pcl_localization | 1.042113 | 0.677669 | 0.364314 | 0.791685 | 0.167265 | 2.895213 |

Table 7.10: Statistics of the 2D APEs for all the three tested 3D Localization Algorithms in the Pitch 2D Localization Challenge in meters

As the 2D localization is of more importance in this application as discussed in 7.1.2, the statistics of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge in meters, given in 7.10, are interpreted as follows:

**rmse** It is the minimum for the pcl\_localization at about 1.04 meters.

**mean** It is the minimum for the pcl\_localization at about 0.68 meters.

**median** It is the minimum for the pcl\_localization at about 0.36 meters.

**std** It is the minimum for the Cartographer at about 0.63 meters.

**min** It is the minimum for the pcl\_localization at about 0.17 meters.

**max** It is the minimum for the pcl\_localization at about 2.9 meters.

## Winner

The Autoware wins in this 3D localization challenge as it is the winner of four out of the six statistics of the 3D APEs. But as discussed in 7.1.2, the 2D localization is of more importance, the pcl\_localization wins in this 2D localization challenge as it is the winner of the five out of the six statistics of the 2D APEs as discussed in 7.2.4.

## 7.2.5 Roll

### Trajectories Comparisons

From 7.33, it is clearly seen that the Autoware and the pcl\_localization were able to quickly recover the pose of the robot after it goes over the slope. But the Cartographer was not able to perform localization in this challenge and it lost its localization estimate.



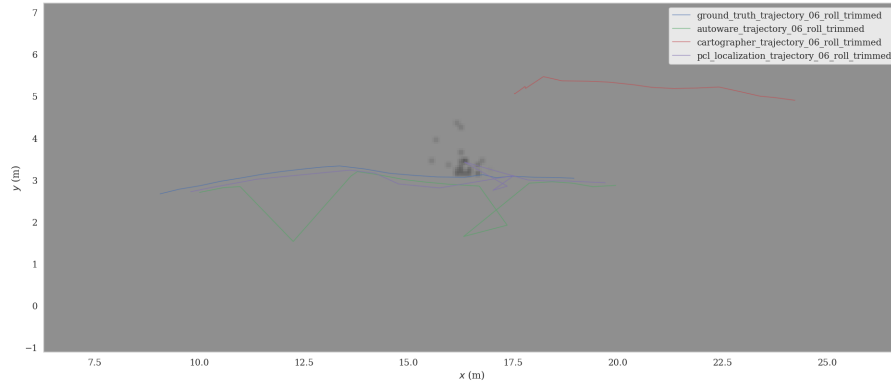


Figure 7.33: Map Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Roll 3D Localization Challenge

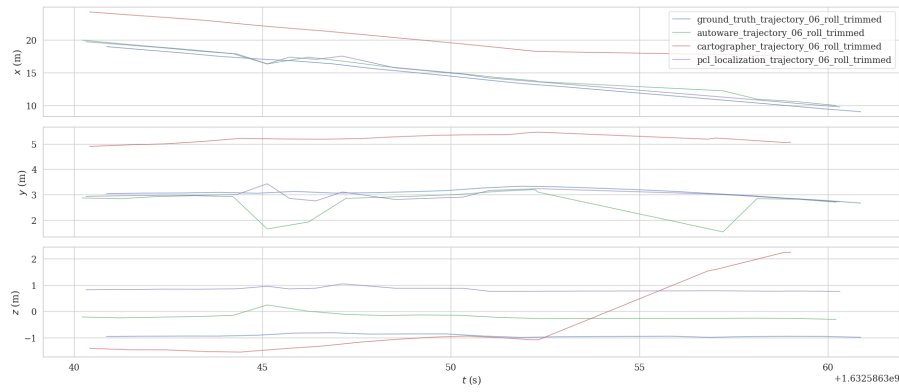


Figure 7.34: Axis-Wise Comparison of the Trajectories of all the three tested 3D Localization Algorithms with the Ground Truth Trajectories in the Roll 3D Localization Challenge

From 7.34, it is seen that the x-axis and the y-axis of the trajectories of the Autoware and the pcl\_localization were disturbed by the slope but they then recovered quickly and were aligned with the x-axis and the y-axis of the trajectory of the ground truth. The x-axis and the y-axis of the trajectory of the Cartographer were not able align with the x-axis and the y-axis of the trajectory of the ground truth. The z-axis of the trajectory of the Autoware and the pcl\_localization are quite parallel to the z-axis of the trajectory of the ground truth and to themselves which is also discussed in 7.2.1. The z-axis in these trajectories are quite parallel to each other, except that of the Cartographer which has a very high range and is inaccurate.

## APEs Distributions

The raw distributions of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge are shown in 7.35. Their interpretation is as follows:

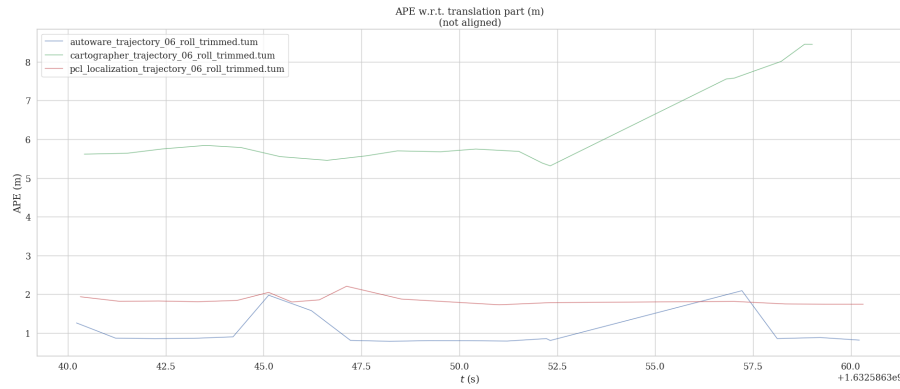


Figure 7.35: Raw Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Roll 3D Localization Challenge

1. The Cartographer has the highest distribution of the 3D APEs among these three tested 3D localization algorithms.
2. The Autoware mostly has the lowest distribution of the 3D APEs among these three tested 3D localization algorithms.
3. The pcl\_localization mostly has the distribution of the 3D APEs between that of the Autoware and the Cartographer.
4. From this plot, one can say that the Autoware is the winner of this 3D localization challenge as it mostly has the lowest distribution of the 3D APEs among these three tested 3D localization algorithms.

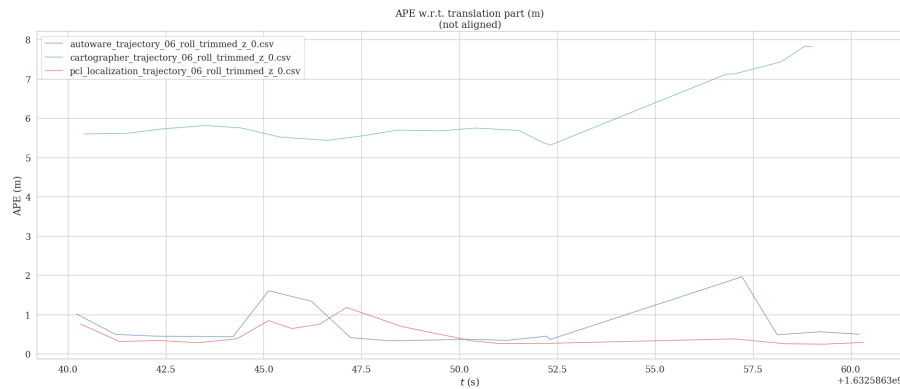


Figure 7.36: Raw Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the Roll 2D Localization Challenge

As the 2D localization is of more importance in this application as discussed in 7.1.2, the raw distributions of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge are shown in 7.36. Their interpretation is as follows:

1. The Cartographer has the highest distribution of the 2D APEs among these three tested 3D localization algorithms.
2. The pcl\_localization mostly has the lowest distribution of the 2D APEs among these three tested 3D localization algorithms.
3. The Autoware mostly has the distribution of the 2D APEs between that of the pcl\_localization and the Cartographer.
4. From this plot, one can say that the pcl\_localization is the winner of this 2D localization challenge as it mostly has the lowest distribution of the 2D APEs among these three tested 3D localization algorithms.

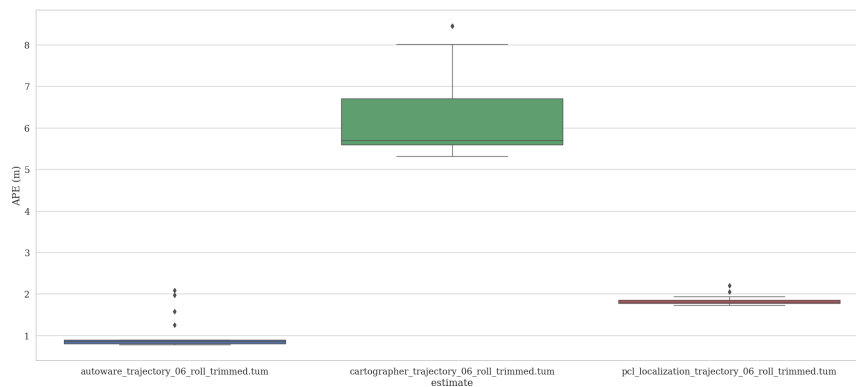


Figure 7.37: Boxplot of the Distributions of the 3D APEs for all the three tested 3D Localization Algorithms in the Roll 3D Localization Challenge

The interpretation of the boxplot of the distributions of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge shown in 7.37 is as follows:

1. The Autoware has the lowest minimum and the Cartographer has the highest maximum 3D APE among these three tested 3D localization algorithms.
2. The range of the distribution of the 3D APEs excluding the outliers is the smallest for the Autoware and the largest for the Cartographer among these three tested 3D localization algorithms.
3. The distribution of the 3D APEs for the Cartographer is positively skewed, which depicts that its distribution of the 3D APEs is concentrated on the left (or below) of the figure.
4. As visually perceived from this boxplot, the Autoware performs the best 3D localization among these three tested 3D localization algorithms in this 3D localization challenge as it has the lowest minimum and the lowest maximum 3D APE and the smallest distribution of the 3D APEs excluding the outliers among them.

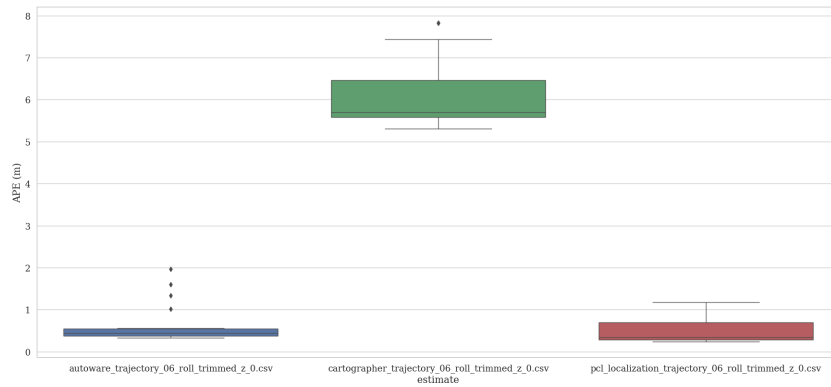


Figure 7.38: Boxplot of the Distributions of the 2D APEs for all the three tested 3D Localization Algorithms in the Roll 2D Localization Challenge

| Roll 3D          | rmse     | mean     | median   | std      | min      | max      |
|------------------|----------|----------|----------|----------|----------|----------|
| Autoware         | 1.069444 | 0.999082 | 0.852652 | 0.381505 | 0.783492 | 2.090645 |
| Cartographer     | 6.25296  | 6.162174 | 5.687072 | 1.06166  | 5.314099 | 8.453413 |
| pcl_localization | 1.83418  | 1.830911 | 1.798627 | 0.109469 | 1.726156 | 2.2051   |

Table 7.11: Statistics of the 3D APEs for all the three tested 3D Localization Algorithms in the Roll 3D Localization Challenge in meters

As the 2D localization is of more importance as discussed in 7.1.2, the distributions of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge are shown in shown in 7.38. Their interpretation is as follows:

1. The pcl\_localization has the lowest minimum and the Autoware has the lowest maximum 2D APE among these three tested 3D localization algorithms.
2. The range of the distribution of the 2D APEs excluding the outliers is the smallest for the Autoware and the largest for the Cartographer among these three tested 3D localization algorithms.
3. The distribution of the 2D APEs for all these three 3D tested localization algorithms is positively skewed, which depicts that they are concentrated on the left (or below) of the figure.
4. Overall it is hard to decide a winner between these three tested 3D localization algorithms in this 2D localization challenge by just visualizing this boxplot.

| Roll 2D          | rmse     | mean     | median   | std      | min      | max      |
|------------------|----------|----------|----------|----------|----------|----------|
| Autoware         | 0.763193 | 0.617802 | 0.438826 | 0.448089 | 0.330709 | 1.960734 |
| Cartographer     | 6.08029  | 6.022819 | 5.687021 | 0.834014 | 5.313334 | 7.827986 |
| pcl_localization | 0.509227 | 0.440707 | 0.311085 | 0.255126 | 0.243694 | 1.179064 |

Table 7.12: Statistics of the 2D APEs for all the three tested 3D Localization Algorithms in the Roll 2D Localization Challenge in meters

### APEs Statistics

The statistics of the 3D APEs for all the three tested 3D localization algorithms in this 3D localization challenge in meters, given in 7.11, are interpreted as follows:

**rmse** It is the minimum for the Autoware at about 1.07 meters.

**mean** It is the minimum for the Autoware at about 1.0 meters.

**median** It is the minimum for the Autoware at about 0.85 meters.

**std** It is the minimum for the pcl\_localization at about 0.11 meters.

**min** It is the minimum for the Autoware at about 0.78 meters.

**max** It is the minimum for the Autoware at about 2.09 meters.

As the 2D localization is of more importance as discussed in 7.1.2, the statistics of the 2D APEs for all the three tested 3D localization algorithms in this 2D localization challenge in meters, given in 7.12, are interpreted as follows:

**rmse** It is the minimum for the pcl\_localization at about 0.51 meters.

**mean** It is the minimum for the pcl\_localization at about 0.44 meters.

**median** It is the minimum for the pcl\_localization at about 0.31 meters.

**std** It is the minimum for the pcl\_localization at about 0.26 meters.

**min** It is the minimum for the pcl\_localization at about 0.24 meters.

**max** It is the minimum for the pcl\_localization at about 1.18 meters.

## Winner

The Autoware is the winner of this 3D localization challenge as it is the winner of five out of the six statistics of the 3D APEs. But as discussed in 7.1.2, the 2D localization is of more importance, the pcl\_localization is the winner of this 2D localization challenge as it is the winner of all of the six statistics of the 2D APEs as discussed in 7.2.5.

### 7.2.6 Overall Winner

As the Autoware won all the five 3D localization challenges as discussed above, it is the overall winner of all the three tested 3D localization algorithms. But as discussed in 7.1.2, the 2D localization is of more importance in this application, the pcl\_localization is the overall winner of all the three tested 3D localization algorithms. This is because it won in three out of the five 2D localization challenges. Therefore, the pcl\_localization is the best performing localization algorithm among the three tested 3D localization algorithms in this application. The Cartographer won the other two 2D localization challenges and therefore is the second best localization algorithm among the three tested 3D localization algorithms in this application. The Autoware did not win any 2D localization challenge and therefore is the least performing localization algorithm among the three tested 3D localization algorithms in this application. Although, if the use case would have been 3D localization, then the Autoware would have been the clear winner of all the three tested 3D localization algorithms in this application as it won all of the five 3D localization challenges as discussed above.

## 8 Conclusions and Future Work

As shown in 7, the quality of the localization is heavily dependent on the quality of the map. Far away from the origin of the map, the quality of the localization decreases due to the deteriorating quality of the map. Therefore, the best map should be created which has the map features equivalent to the real world map. The ways to improve the quality of the map are:

1. The map quality could be improved by recording the dataset made by the robot closing the loop in the map as discussed in 6.2.4. This could correct for the drift in the map observed as curved walls as seen in 7.1.
2. If the Cartographer SLAM algorithm is used for making the map, then the following steps can be performed for improving the quality of the localization in this application:
  - a) One way could be setting the parameter `POSE_GRAPH.optimization_problem.odometry_rotation_weight` to zero. It is because the odometry of a tracked wheel robot is not accurate when it is rotating as discussed in 6.2.11.
  - b) One could also include ground truth RTK GNSS data as a sensor input to this SLAM algorithm for improving the quality of its global SLAM as mentioned in [22]. More about the global SLAM in this algorithm is discussed in 6.2.4. Though in this application, the ground truth RTK GNSS data was not used as an input for this SLAM algorithm, as it was not very accurate because it was noisy and had many outliers, which was discussed in 7.1.
3. One more way to improve the quality of the map is to perform mapping instead of SLAM as discussed in 6.2.4.

As discussed in 7, the `pcl_localization` performs the best 2D localization among the three tested 3D localization algorithms. The Cartographer is at the second place and the Autoware is at the third place in performing the 2D localization. Though if the 3D localization was the use case, then the Autoware performs the best 3D localization among the three tested 3D localization algorithms. All these three tested 3D localization algorithms were tuned as much as possible in the six months duration of this master thesis to achieve these results. Also, if some parameter tuning had no effect or if it worsened the quality of the localization then the default parameters were used for that 3D localization algorithm.

Further parameter tuning could improve the quality of the localization in these three tested 3D localization algorithms. For example, one possible way for tuning the Cartographer could be to set `POSE_GRAPH.optimization_problem.odometry_rotation_weight` to zero as discussed above. Increasing the amount of sensor data input could also improve the quality of the localization for a localization algorithm. For example, if a nine-axis IMU is available which also provides an orientation estimate, it can be used as a sensor data input for the `pcl_localization` to improve its quality of localization as discussed in 6.1.



# Bibliography

- [1] "PCAN-USB." <https://www.peak-system.com/PCAN-USB.199.0.html?&L=1>. Accessed: 2021-07-07.
- [2] D. Thomas, W. Woodall, and E. Fernandez, "Next-generation ROS: Building on DDS," in *ROSCon Chicago 2014*, (Mountain View, CA), Open Robotics, sep 2014.
- [3] M. Quigley, B. Gerkey, K. Conley, J. Faust, T. Foote, J. Leibs, E. Berger, R. Wheeler, and A. Ng, "Ros: an open-source robot operating system," in *Proc. of the IEEE Intl. Conf. on Robotics and Automation (ICRA) Workshop on Open Source Robotics*, (Kobe, Japan), may 2009.
- [4] "InMach Intelligente Maschinen GmbH." <https://www.inmach.de/de/>. Accessed: 2021-07-12.
- [5] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The marathon 2: A navigation system," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020.
- [6] S. Macenski and I. Jambrecic, "Slam toolbox: Slam for the dynamic world," *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.
- [7] S. Macenski, "On use of slam toolbox," in *ROSCon Macau 2019*, Open Robotics, October 2019.
- [8] C. Stachniss, "Robot Mapping Scan-Matching in 5 Minutes." <http://ais.informatik.uni-freiburg.de/teaching/ws12/mapping/pdf/slam12-scanmatching-mini.pdf>. Accessed: 2021-07-16.
- [9] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics (Intelligent Robotics and Autonomous Agents)*. The MIT Press, 2005.
- [10] "pgm." <http://netpbm.sourceforge.net/doc/pgm.html>. Accessed: 2021-10-20.
- [11] "Map Server." [http://wiki.ros.org/map\\_server](http://wiki.ros.org/map_server). Accessed: 2021-07-10.
- [12] "YAML." <https://yaml.org/>. Accessed: 2021-07-25.

- [13] "The PCD (Point Cloud Data) file format." [https://pointclouds.org/documentation/tutorials/pcd\\_file\\_format.html](https://pointclouds.org/documentation/tutorials/pcd_file_format.html). Accessed: 2021-07-30.
- [14] "ROS 2 Documentation." <https://docs.ros.org/en/foxy/index.html>. Accessed: 2021-07-22.
- [15] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2d lidar slam," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1271–1278, 2016.
- [16] P. Biber and W. Strasser, "The normal distributions transform: a new approach to laser scan matching," in *Proceedings 2003 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS 2003) (Cat. No.03CH37453)*, vol. 3, pp. 2743–2748 vol.3, Oct 2003.
- [17] M. Magnusson, *The Three-Dimensional Normal-Distributions Transform – an Efficient Representation for Registration, Surface Analysis, and Loop Detection*. PhD thesis, 12 2009.
- [18] S. Kato, S. Tokunaga, Y. Maruyama, S. Maeda, M. Hirabayashi, Y. Kitsukawa, A. Monrroy, T. Ando, Y. Fujii, and T. Azumi, "Autoware on board: Enabling autonomous vehicles with embedded systems," in *Proceedings of the 9th ACM/IEEE International Conference on Cyber-Physical Systems, ICCPS'18*, p. 287–296, IEEE Press, 2018.
- [19] S. Kato, E. Takeuchi, Y. Ishiguro, Y. Ninomiya, K. Takeda, and T. Hamada, "An open approach to autonomous vehicles," *IEEE Micro*, vol. 35, pp. 60–68, Nov 2015.
- [20] R. Sasasaki, "A ROS 2 package of 3D LIDAR-based Localization using PCL." [https://github.com/rsasaki0109/pcl\\_localization\\_ros2](https://github.com/rsasaki0109/pcl_localization_ros2). Accessed: 2021-08-24.
- [21] "Google Cartographer pose extrapolator component." [https://github.com/ros2/cartographer/blob/ros2/cartographer/mapping/pose\\_extrapolator.h](https://github.com/ros2/cartographer/blob/ros2/cartographer/mapping/pose_extrapolator.h). Accessed: 2021-08-22.
- [22] "Cartographer ROS Integration." <https://google-cartographer-ros.readthedocs.io/en/latest/>. Accessed: 2021-07-29.
- [23] "pcl\_localization\_ros2 lidar undistortion component." [https://github.com/rsasaki0109/pcl\\_localization\\_ros2/blob/master/include/pcl\\_localization/lidar\\_undistortion.hpp](https://github.com/rsasaki0109/pcl_localization_ros2/blob/master/include/pcl_localization/lidar_undistortion.hpp). Accessed: 2021-08-26.
- [24] "MATPRO ROVO 2." <https://www.mattro.com/en/rovo/>. Accessed: 2021-07-01.
- [25] "Logitech F710 Wireless Gamepad." <https://www.logitechg.com/en-us/products/gamepads/f710-wireless-gamepad.940-000117.html>. Accessed: 2021-07-03.

- [26] "Lenovo ThinkPad W541 Mobile Workstation." <https://www.lenovo.com/gb/en/laptops/thinkpad/w-series/w541/>. Accessed: 2021-07-04.
- [27] "Intel® Core™ i7-4810MQ Processor." <https://ark.intel.com/content/www/us/en/ark/products/78937/intel-core-i74810mq-processor-6m-cache-up-to-3-80-ghz.html>. Accessed: 2021-07-04.
- [28] "NVIDIA PROFESSIONAL GRAPHICS SOLUTIONS." [https://www.nvidia.com/content/PDF/line\\_card/6660-nv-prographicssolutions-linecard-july13-final-lr.pdf](https://www.nvidia.com/content/PDF/line_card/6660-nv-prographicssolutions-linecard-july13-final-lr.pdf). Accessed: 2021-07-05.
- [29] "Download Ubuntu Desktop." <https://ubuntu.com/download/desktop>. Accessed: 2021-07-29.
- [30] "Ouster OS1 lidar sensor." <https://ouster.com/products/os1-lidar-sensor/>. Accessed: 2021-07-02.
- [31] "Conventions for IMU Sensor Drivers." <https://www.ros.org/reps/rep-0145.html>. Accessed: 2021-08-19.
- [32] "Emlid Reach M+." <https://emlid.com/reach/>. Accessed: 2021-08-30.
- [33] "Swift Navigation Mini Survey Antenna GPS500." <https://www.swiftnav.com/resource-files/GNSS%20Antenna/v1.2/Specification/SwiftNav%20Mini%20Survey%20Antenna%20GPS500%20Specifications%20v1.2.pdf>. Accessed: 2021-08-31.
- [34] "Satellitenpositionierungsdienst (SAPOS®) Baden-Württemberg." <https://www.sapos-bw.de/>. Accessed: 2021-09-01.
- [35] "ROS Joystick Drivers Stack." [https://github.com/ros-drivers/joystick\\_drivers/tree/ros2](https://github.com/ros-drivers/joystick_drivers/tree/ros2). Accessed: 2021-07-08.
- [36] "Joy Message." [https://docs.ros2.org/foxy/api/sensor\\_msgs/msg/Joy.html](https://docs.ros2.org/foxy/api/sensor_msgs/msg/Joy.html). Accessed: 2021-07-10.
- [37] B. Stroustrup, *The C++ programming language*. Second edition, reprinted with corrections August, 1995. Reading, Mass. : Addison-Wesley, 1995. ©1991, 1995. Includes bibliographical references (pages 11-12) and index.
- [38] "Odometry Message." [https://docs.ros2.org/foxy/api/nav\\_msgs/msg/Odometry.html](https://docs.ros2.org/foxy/api/nav_msgs/msg/Odometry.html). Accessed: 2021-07-14.

- [39] "Coordinate Frames for Mobile Platforms." <https://www.ros.org/reps/rep-0105.html>. Accessed: 2021-07-13.
- [40] "PoseWithCovariance Message." [https://docs.ros2.org/foxy/api/geometry\\_msgs/msg/PoseWithCovariance.html](https://docs.ros2.org/foxy/api/geometry_msgs/msg/PoseWithCovariance.html). Accessed: 2021-10-10.
- [41] "Pose Message." [https://docs.ros2.org/foxy/api/geometry\\_msgs/msg/Pose.html](https://docs.ros2.org/foxy/api/geometry_msgs/msg/Pose.html). Accessed: 2021-11-11.
- [42] "Point Message." [https://docs.ros2.org/foxy/api/geometry\\_msgs/msg/Point.html](https://docs.ros2.org/foxy/api/geometry_msgs/msg/Point.html). Accessed: 2021-10-18.
- [43] "Quaternion Message." [https://docs.ros2.org/foxy/api/geometry\\_msgs/msg/Quaternion.html](https://docs.ros2.org/foxy/api/geometry_msgs/msg/Quaternion.html). Accessed: 2021-08-26.
- [44] "TwistWithCovariance Message." [https://docs.ros2.org/foxy/api/geometry\\_msgs/msg/TwistWithCovariance.html](https://docs.ros2.org/foxy/api/geometry_msgs/msg/TwistWithCovariance.html). Accessed: 2021-09-17.
- [45] "Twist Message." [https://docs.ros2.org/foxy/api/geometry\\_msgs/msg/Twist.html](https://docs.ros2.org/foxy/api/geometry_msgs/msg/Twist.html). Accessed: 2021-08-17.
- [46] "Vector3 Message." [https://docs.ros2.org/foxy/api/geometry\\_msgs/msg/Vector3.html](https://docs.ros2.org/foxy/api/geometry_msgs/msg/Vector3.html). Accessed: 2021-07-19.
- [47] "ROS 2 Ouster Drivers." [https://github.com/ros-drivers/ros2\\_ouster\\_drivers](https://github.com/ros-drivers/ros2_ouster_drivers). Accessed: 2021-07-12.
- [48] "TFMessage." [https://docs.ros2.org/foxy/api/tf2\\_msgs/msg/TFMessage.html](https://docs.ros2.org/foxy/api/tf2_msgs/msg/TFMessage.html). Accessed: 2021-08-20.
- [49] "Ouster Software User Manual." <https://data.ouster.io/downloads/software-user-manual/software-user-manual-v2.1.x.pdf>. Accessed: 2021-08-29.
- [50] "Coordinate frame transformations different from the Ouster software manual." [https://github.com/ros-drivers/ros2\\_ouster\\_drivers/issues/84](https://github.com/ros-drivers/ros2_ouster_drivers/issues/84). Accessed: 2021-08-12.
- [51] "PointCloud2 Message." [https://docs.ros2.org/foxy/api/sensor\\_msgs/msg/PointCloud2.html](https://docs.ros2.org/foxy/api/sensor_msgs/msg/PointCloud2.html). Accessed: 2021-08-18.
- [52] "Imu Message." [https://docs.ros2.org/foxy/api/sensor\\_msgs/msg/Imu.html](https://docs.ros2.org/foxy/api/sensor_msgs/msg/Imu.html). Accessed: 2021-08-15.

- [53] "NMEA navsat driver." [https://github.com/ros-drivers/nmea\\_navsat\\_driver/tree/ros2](https://github.com/ros-drivers/nmea_navsat_driver/tree/ros2). Accessed: 2021-08-11.
- [54] "NavSatFix Message." [https://docs.ros2.org/foxy/api/sensor\\_msgs/msg/NavSatFix.html](https://docs.ros2.org/foxy/api/sensor_msgs/msg/NavSatFix.html). Accessed: 2021-09-21.
- [55] "NMEA Revealed." <https://gpsd.gitlab.io/gpsd/NMEA.html>. Accessed: 2021-09-17.
- [56] "NavSatStatus Message." [https://docs.ros2.org/foxy/api/sensor\\_msgs/msg/NavSatStatus.html](https://docs.ros2.org/foxy/api/sensor_msgs/msg/NavSatStatus.html). Accessed: 2021-09-05.
- [57] "World Geodetic System 1984." [https://www.unoosa.org/pdf/icg/2012/template/WGS\\_84.pdf](https://www.unoosa.org/pdf/icg/2012/template/WGS_84.pdf). Accessed: 2021-09-24.
- [58] "Conversion of Geodetic coordinates to the Local Tangent Plane." [http://archive.psas.pdx.edu/CoordinateSystem/Latitude\\_to\\_LocalTangent.pdf](http://archive.psas.pdx.edu/CoordinateSystem/Latitude_to_LocalTangent.pdf). Accessed: 2021-09-26.
- [59] "RMC - Recommended Minimum Navigation Information." [https://gpsd.gitlab.io/gpsd/NMEA.html#\\_rmc\\_recommended\\_minimum\\_navigation\\_information](https://gpsd.gitlab.io/gpsd/NMEA.html#_rmc_recommended_minimum_navigation_information). Accessed: 2021-08-20.
- [60] "Rosbag2." <https://github.com/ros2/rosbag2>. Accessed: 2021-09-06.
- [61] "rqt\_graph." [https://github.com/ros-visualization/rqt\\_graph/tree/foxy-devel](https://github.com/ros-visualization/rqt_graph/tree/foxy-devel). Accessed: 2021-08-30.
- [62] "CAN Frame Message." [https://github.com/ros-industrial/ros\\_canopen/blob/dashing/can\\_msgs/msg/Frame.msg](https://github.com/ros-industrial/ros_canopen/blob/dashing/can_msgs/msg/Frame.msg). Accessed: 2021-08-17.
- [63] "rqt\_topic." [https://github.com/ros-visualization/rqt\\_topic/tree/foxy-devel](https://github.com/ros-visualization/rqt_topic/tree/foxy-devel). Accessed: 2021-08-09.
- [64] "Foxy Improvement Backport Branch." <https://github.com/ros2/rosbag2/tree/foxy-future>. Accessed: 2021-09-13.
- [65] "Standard Units of Measure and Coordinate Conventions." <https://www.ros.org/reps/rep-0103.html>. Accessed: 2021-07-30.
- [66] S. Agarwal, K. Mierle, and Others, "Ceres solver." <http://ceres-solver.org>.
- [67] "Google Cartographer motion filter component." <https://github.com/ros2/>

- cartographer/blob/ros2/cartographer/mapping/internal/motion\_filter.h. Accessed: 2021-08-06.
- [68] "Google Cartographer IMU tracker component." [https://github.com/ros2/cartographer/blob/ros2/cartographer/mapping/imu\\_tracker.h](https://github.com/ros2/cartographer/blob/ros2/cartographer/mapping/imu_tracker.h). Accessed: 2021-09-09.
- [69] "Lua." <https://www.lua.org/>. Accessed: 2021-07-27.
- [70] "Why Google Cartographer assets writer has been disabled for ROS 2." [https://github.com/ros2/cartographer\\_ros/issues/47](https://github.com/ros2/cartographer_ros/issues/47). Accessed: 2021-08-01.
- [71] "Merge cloud from pbstream." [https://github.com/cyy1992/temp\\_ws/blob/26f05d364b433c04d8cee30dd8c8da1b6c8efcc7/src/test6/src/merge\\_cloud\\_from\\_pbstream.h](https://github.com/cyy1992/temp_ws/blob/26f05d364b433c04d8cee30dd8c8da1b6c8efcc7/src/test6/src/merge_cloud_from_pbstream.h). Accessed: 2021-08-02.
- [72] R. B. Rusu and S. Cousins, "3D is here: Point Cloud Library (PCL)," in *IEEE International Conference on Robotics and Automation (ICRA)*, (Shanghai, China), IEEE, May 9-13 2011.
- [73] "Cartographer." <https://google-cartographer.readthedocs.io/en/latest/>. Accessed: 2021-07-28.
- [74] "GeographicLib::LocalCartesian Class Reference." [https://geographiclib.sourceforge.io/html/classGeographicLib\\_1\\_1LocalCartesian.html](https://geographiclib.sourceforge.io/html/classGeographicLib_1_1LocalCartesian.html). Accessed: 2021-08-18.
- [75] C. F. F. Karney, "GeographicLib, Version 1.52 (2021-06-22)." <https://geographiclib.sourceforge.io/1.52/>. Accessed: 2021-08-23.
- [76] "EPSG:25832 ETRS89 / UTM zone 32N." <https://epsg.io/25832>. Accessed: 2021-10-06.
- [77] "GIMP (GNU IMAGE MANIPULATION PROGRAM)." <https://www.gimp.org/>. Accessed: 2021-10-09.
- [78] "Google Maps." <https://www.google.com/maps/@48.3654395,9.9400892,148a,35y,142.27h/data=!3m1!1e3>. Accessed: 2021-08-11.
- [79] "Google Cartographer pbstream to ros map component." [https://github.com/ros2/cartographer\\_ros/blob/dashing/cartographer\\_ros/cartographer\\_ros/pbstream\\_to\\_ros\\_map\\_main.cc](https://github.com/ros2/cartographer_ros/blob/dashing/cartographer_ros/cartographer_ros/pbstream_to_ros_map_main.cc). Accessed: 2021-08-07.
- [80] M. Grupp, "evo: Python package for the evaluation of odometry and SLAM." <https://github.com/MichaelGrupp/evo>, 2017.

- [81] S. Umeyama, "Least-squares estimation of transformation parameters between two point patterns," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, pp. 376–380, April 1991.
- [82] "File Formats." [https://vision.in.tum.de/data/datasets/rgbd-dataset/file\\_formats](https://vision.in.tum.de/data/datasets/rgbd-dataset/file_formats). Accessed: 2021-10-24.
- [83] "Google Cartographer pbstream trajectories to rosbag." [https://github.com/cartographer-project/cartographer\\_ros/blob/master/cartographer\\_ros/cartographer\\_ros/dev/pbstream\\_trajectories\\_to\\_rosbag\\_main.cc](https://github.com/cartographer-project/cartographer_ros/blob/master/cartographer_ros/cartographer_ros/dev/pbstream_trajectories_to_rosbag_main.cc). Accessed: 2021-08-21.
- [84] "RViz." <https://github.com/ros2/rviz>. Accessed: 2021-07-25.
- [85] "ISO 10303-21:2016." <https://www.iso.org/standard/63141.html>. Accessed: 2021-07-06.
- [86] "What Is An STL File?." <https://www.3dsystems.com/quickparts/learning-center/what-is-stl-file>. Accessed: 2021-07-07.
- [87] "FreeCAD." <https://www.freecadweb.org/>. Accessed: 2021-07-01.
- [88] "COLLADA™." <https://www.khronos.org/api/collada>. Accessed: 2021-07-08.
- [89] "Blender." <http://www.blender.org>. Accessed: 2021-07-10.
- [90] "STL vs DAE." <https://answers.ros.org/question/39959/stl-vs-dae/>. Accessed: 2021-07-12.
- [91] "URDF." <https://github.com/ros2/urdf>. Accessed: 2021-07-13.
- [92] "Extensible Markup Language (XML)." <https://www.w3.org/XML/>. Accessed: 2021-07-16.
- [93] "STEP files for URDF model." <https://answers.ros.org/question/262253/step-files-for-urdf-model/>. Accessed: 2021-07-11.
- [94] "Using URDF with robot state publisher." [https://github.com/ros/robot\\_state\\_publisher/tree/ros2](https://github.com/ros/robot_state_publisher/tree/ros2). Accessed: 2021-07-21.
- [95] Wikipedia contributors, "Box plot — Wikipedia, the free encyclopedia." [https://en.wikipedia.org/w/index.php?title=Box\\_plot&oldid=1059977055](https://en.wikipedia.org/w/index.php?title=Box_plot&oldid=1059977055), 2021. [Online; accessed 12-December-2021].

[96] Michael Galarnyk, "Understanding Boxplots." <https://towardsdatascience.com/understanding-boxplots-5e2df7bcbd51>. Accessed: 2021-12-12.